Abdelmalek Essaâdi University
National School of Applied Sciences of Al Hoceima (ENSAH)

Module: Cloud Computing, DataOps and MLOps

Workshop 2: OpenStack

Deploying and Administering a Private Cloud

Prepared by:

Yousra Khallou
Oumaima El Ghalbouni
Firdawss El Haddouchi

Supervised by:

Pr Hayat Routaib

Class: ID3
Academic year 2026/2027

Contents

List of Figures **3**

List of Tables **6**

Context and Problem Statement **7**

1 Part 1 – Preliminary Study and Environment Preparation **7**
1.1 Overview of OpenStack and Its Value for a Private Cloud 7
1.2 Main OpenStack Components 8
1.3 Role of Each Component in the Architecture 8
1.4 Characteristics of the Machine Used 10
1.5 Choice of the Deployment Method 13

2 Part 2 – Installing OpenStack **16**
2.1 Preparing the Controller Node 17
2.2 Preparing the compute1 Node 18
2.3 Preparing the compute2 node 20
2.4 First networking method: Wi-Fi in bridged mode 22
2.5 Limitations of the first method 27
2.6 Solution: the Tailscale virtual private network 28
2.7 Installing the controller node 30
2.8 Installing the compute nodes 31
2.9 Verifying the services 35
2.10 Accessing the Horizon administration interface 38
2.11 Difficulties encountered and solutions adopted 40

3 Part 3 – Creating and administering cloud resources **42**
3.1 System image 44
3.2 Instance type (flavor) 45
3.3 Private network and subnet 45
3.4 Virtual router and external access 46
3.5 SSH key pair 47
3.6 Security rules 47
3.7 Launching instances 48
3.8 Checking the state and resources of the instances 49

4 Part 4 – Testing and validation **50**
4.1 Instance started correctly 50

4.2 Network connectivity test 51
4.3 SSH connection to the instance 52
4.4 Enforcement of the security rules 53
4.5 Stopping, restarting and then deleting an instance 53
4.6 Resource monitoring 54

5 Conclusion **55**

List of Figures

1 Simplified relationships between the main OpenStack components (Keystone authenticates the requests sent to all services) . . . 10
2 Virtual machine configuration in VirtualBox . . . 11
3 Characteristics of the physical PC (Windows) . . . 11
4 Characteristics of the Ubuntu VM collected from the command line . . . 12
5 No hardware virtualization on `compute2` . . . 12
6 Hardware requirements given in the official OpenStack documentation . . . 15
7 Deployed multi-node architecture: one controller and two compute nodes linked by a VPN . . . 16
8 Machine name, clock, firewall and `stack` user of the controller . . . 17
9 Fetching DevStack and system version of the controller . . . 18
10 System version of `compute1` after the upgrade (Ubuntu 24.04.5 LTS) . . . 19
11 System update and git installation on `compute1` . . . 19
12 Creation of the `stack` user on `compute1` . . . 20
13 Fetching DevStack on `compute1` (branch `stable/2026.1`) . . . 20
14 System version of `compute2` (Ubuntu 24.04.5 LTS) . . . 21
15 Hostname of `compute2` . . . 21
16 `stack` user on `compute2` . . . 21
17 git installation on `compute2` . . . 22
18 Fetching DevStack on `compute2` . . . 22
19 VirtualBox network adapter of the controller in “Bridged Adapter” mode . . . 23
20 VirtualBox network adapter of `compute1` in “Bridged Adapter” mode . . . 23
21 Fixed IP address of the controller . . . 23
22 Fixed IP address of `compute1` and Internet access . . . 24
23 Fixed IP address of `compute2` . . . 24
24 `/etc/hosts` file of the controller . . . 24
25 `/etc/hosts` file of `compute1` . . . 25
26 `/etc/hosts` file of `compute2` . . . 25
27 Connectivity tests from the controller to `compute1` and `compute2` . . . 26
28 Connectivity tests from `compute1` to the controller and compute2 . . . 26
29 Connectivity tests from `compute2` to `compute1` and the controller . . . 27
30 Network change of the controller (subnets `192.168.1.0/24` then `192.168.0.0/24`) . . . 28
31 Tailscale VPN status and connectivity tests between the nodes (view from `compute1`) . . . 29
32 Connectivity test from `compute2` to the controller through the VPN . . . 29
33 Tailscale VPN status and connectivity tests seen from the controller . . . 30

34 VPN status seen from the controller: connection to `compute1` through a relay, direct connection to `compute2` . . . 30
35 `local.conf` file of the `compute1` node . . . 32
36 End of the `compute1` installation (`stack.sh` summary) . . . 33
37 `devstack@n-cpu` service active on `compute1` . . . 33
38 `local.conf` file of the `compute2` node . . . 34
39 End of the `compute2` installation (`stack.sh` summary) . . . 34
40 `devstack@n-cpu` service active on `compute2` . . . 35
41 Compute services: controller, `compute1` and `compute2` in the `up` state . . . 36
42 Hypervisor list: `compute1` and `compute2` (QEMU, `up`) . . . 36
43 Services registered in the Keystone catalog (Horizon) . . . 37
44 Registered services (command line, `openstack service list`) . . . 37
45 Compute services in Horizon . . . 37
46 Network agents in Horizon . . . 38
47 Horizon home page after login . . . 38
48 Hypervisors in Horizon: `compute1` node of QEMU type . . . 39
49 VM clock offset by one hour . . . 41
50 Clock synchronized after restarting `systemd-timesyncd` . . . 41
51 Clock not synchronized on `compute2` (NTP service inactive) . . . 41
52 Clock synchronized on `compute2` after correction . . . 42
53 Leftovers of an old installation in `/opt/stack` . . . 42
54 `compute1` in the `down` state and `compute2` in the `up` state in the compute service list . . . 42
55 Commands used to create the Part 3 resources . . . 44
56 CirrOS image in Glance (command line) . . . 44
57 CirrOS image in Horizon . . . 45
58 The `m1.test` flavor . . . 45
59 Network `tp-net` and subnet `tp-subnet` . . . 46
60 The `tp-router` router . . . 47
61 Networks created by DevStack (`private`, `shared` and `public`) . . . 47
62 Key pair `mykey` and protected private key file . . . 47
63 Instance `test1` running on the node `compute1` . . . 49
64 Instances and the nodes that host them . . . 49
65 Instance seen by the hypervisor in the `compute1` VM . . . 50
66 Empty image cache on `compute2` (no instance hosted) . . . 50
67 State of the instance `test1` and CirrOS boot log . . . 51
68 Creation and association of the floating IP address . . . 52
69 `ping` test to the instance's floating address . . . 52
70 SSH connection to the instance `test1` through its floating address . . . 53

71 Stopping, restarting and deleting the instance `test1` 54
72 Resources of the two compute nodes in Horizon 55

List of Tables

1 The main OpenStack components studied in this workshop 8
2 Characteristics of the machine used 10
3 Comparison of the deployment methods considered 13
4 Hardware requirements of the official documentation and resources of our VMs 15
5 Distribution of the nodes within the team 16
6 Difficulties encountered and solutions adopted 40
7 Cloud resources created 43
8 Addressing plan of the private network 46
9 Instances created during the tests 48
10 Summary of the validation tests 50

Context and Problem Statement

Cloud infrastructures provide computing resources on demand: virtual machines, virtual networks and storage. OpenStack is an open-source platform for building and administering a private cloud of the *Infrastructure as a Service* (IaaS) type.

In this workshop, our team sets up an OpenStack environment on personal machines, then demonstrates how the main services of a private cloud work: creating instances, configuring virtual networks, managing images, instance types (*flavors*) and SSH keys, and testing connectivity.

1 Part 1 – Preliminary Study and Environment Preparation

1.1 Overview of OpenStack and Its Value for a Private Cloud

What is OpenStack?

OpenStack is an open-source *cloud computing* platform designed to control large pools of compute, storage and network resources, managed through a web interface, command-line tools or APIs [1]. It turns a set of physical machines into a service in which users can, on demand and in self-service:

- create and delete virtual machines (instances);
- define virtual networks, subnets and routers;
- manage system images, storage volumes and SSH keys.

This model corresponds to **IaaS** (*Infrastructure as a Service*): the user has a virtualized infrastructure that they configure themselves, without having to manage the underlying hardware.

OpenStack is not a monolithic piece of software: it is a set of independent services that communicate with each other through APIs. This modularity makes it possible to install only the services that are needed.

Private cloud and the value of OpenStack

A **private cloud** is a cloud infrastructure operated for a single organization, hosted on its own premises or with a dedicated provider. OpenStack is one of the open-source solutions used to build it. Its value is as follows:

- **Control over resources and data:** the organization controls where its virtual machines and data are hosted, and applies its own security rules.

- **Self-service and resource pooling:** users create their own resources, in separate projects, from the same pool of machines.
- **Open source:** no software licence to pay for, inspectable code, a large community and official documentation.
- **Modularity and extensibility:** only the useful services are enabled, and capacity is increased by adding compute nodes.
- **Automation:** every operation is available through APIs and the command line, which makes automation and integration with other tools easier.

In the context of this workshop, OpenStack allows us to reproduce, on a small scale, the architecture of an enterprise cloud on our own machines, in a test environment.

1.2 Main OpenStack Components

The workshop statement asks for seven OpenStack services. Table 1 identifies them with their general function.

Component	Service type	General function
Keystone	Identity	Authentication, users, projects and access rights
Nova	Compute	Creation and management of virtual machines
Neutron	Networking	Virtual networks, routers, floating IPs, security rules
Glance	Images	Catalogue of system images
Cinder	Block storage	Storage volumes that can be attached to instances
Horizon	Dashboard	Web administration interface
Placement	Resources	Inventory and tracking of available resources

Table 1: The main OpenStack components studied in this workshop

1.3 Role of Each Component in the Architecture

Keystone – identity service

Keystone is the security entry point of the cloud. It authenticates users and services, issues access *tokens*, manages projects (*tenants*), roles and rights, and maintains the *service catalogue* that indicates where to reach the APIs of the other components. Every request sent to the other services goes through a check with Keystone.

Nova – compute service

Nova manages the life cycle of virtual machines: creation, start, stop, reboot, resize and deletion. It receives the user's requests, chooses on which compute node to place the instance, and then the `nova-compute` service installed on that node starts it using a hypervisor (KVM or QEMU). Nova relies on Glance for the image, on Neutron for the network and on Placement for choosing the node.

Neutron – networking service

Neutron provides networking as a service. It makes it possible to create private networks and subnets with an addressing plan, virtual routers, floating IPs for access from outside, and security groups that filter incoming and outgoing traffic of the instances (for example to allow ICMP and SSH).

Glance – image service

Glance stores and distributes virtual machine images (for example Ubuntu or CirrOS) from which Nova creates instances. It also records the metadata of each image.

Cinder – block storage service

Cinder provides persistent volumes, which can be attached to an instance and then detached or moved to another one. Unlike the ephemeral disk of an instance, a Cinder volume survives the deletion of the instance. This service is presented here for the study of the architecture; it is not deployed in our environment (see section 1.5).

Horizon – dashboard

Horizon is the web interface of OpenStack. It makes it possible to administer the cloud (projects, users, images, flavors) and to manage resources (instances, networks, SSH keys) without using the command line. Horizon simply relies on the APIs of the other services.

Placement – resource tracking

Placement keeps the inventory of available resources (processors, memory, disk) on each compute node and of their consumption. Nova queries it to know which nodes can host a new instance.

Overview

Figure 1 outlines the main relationships between these components. It is simplified: it does not show the database or the message queue used by the services.

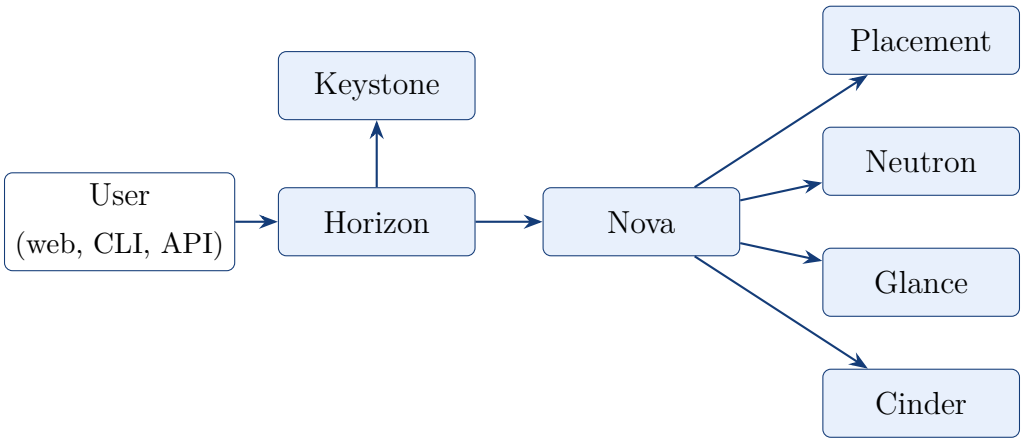


Figure 1: Simplified relationships between the main OpenStack components (Keystone authenticates the requests sent to all services)

1.4 Characteristics of the Machine Used

Our environment has three levels: the **physical PC** running Windows (the VirtualBox host), the **Ubuntu virtual machine** that hosts the OpenStack node, and the **instances** created by OpenStack inside that virtual machine. We recorded the characteristics of the physical PC and of the `compute1` virtual machine.

Item	**Physical PC (host)**	**Ubuntu VM `compute1`**
Operating system	Windows 11	Ubuntu 24.04.5 LTS
Processor	Intel Core i5-10210U, 1.60 GHz	4 virtual processors (vCPU)
RAM	24 GB	16 GB allocated, 15 GB visible in the system
Disk space	256 GB	55 GB virtual disk (54 GB visible)
Hardware virtualization	enabled	Not exposed to the VM (see below)

Table 2: Characteristics of the machine used

Commands used in the VM

The characteristics of the virtual machine were collected with the following commands:

```
LANG=C lscpu | grep -E "Model name|^CPU\(s\)|Virtualization|Hypervisor"
free -h
df -h /
```

```
lsb_release -a
grep -E -c '(vmx|svm)' /proc/cpuinfo
```

They report, respectively, the processor, the memory, the disk space, the operating system version and the presence of the processor's hardware virtualization extensions (`vmx` for Intel VT-x, `svm` for AMD-V).

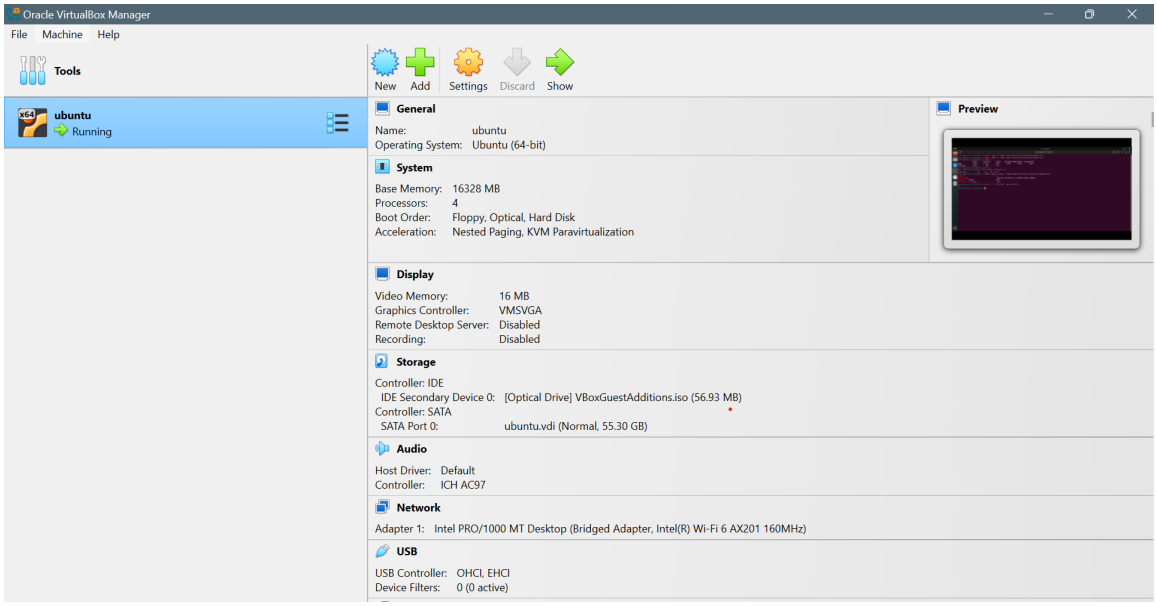


Figure 2: Virtual machine configuration in VirtualBox

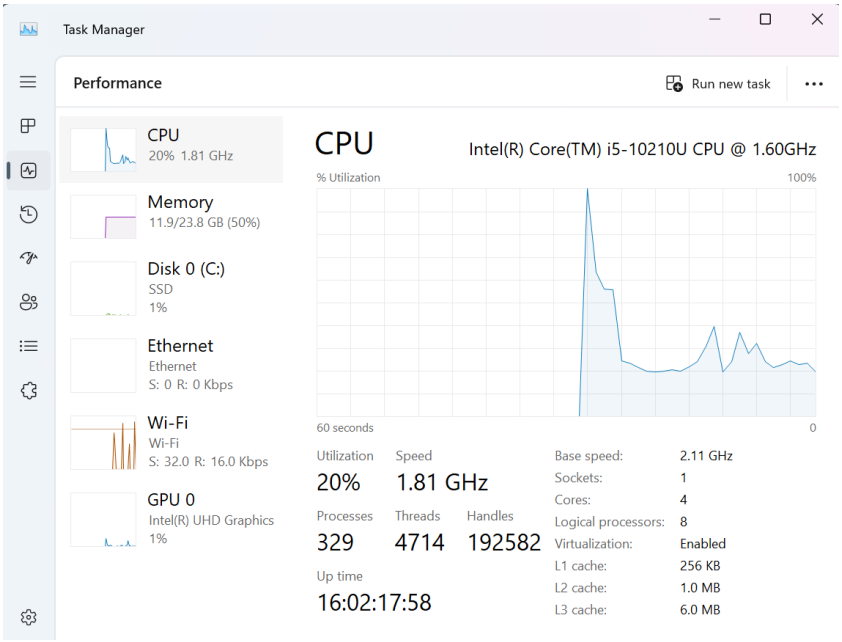


Figure 3: Characteristics of the physical PC (Windows)

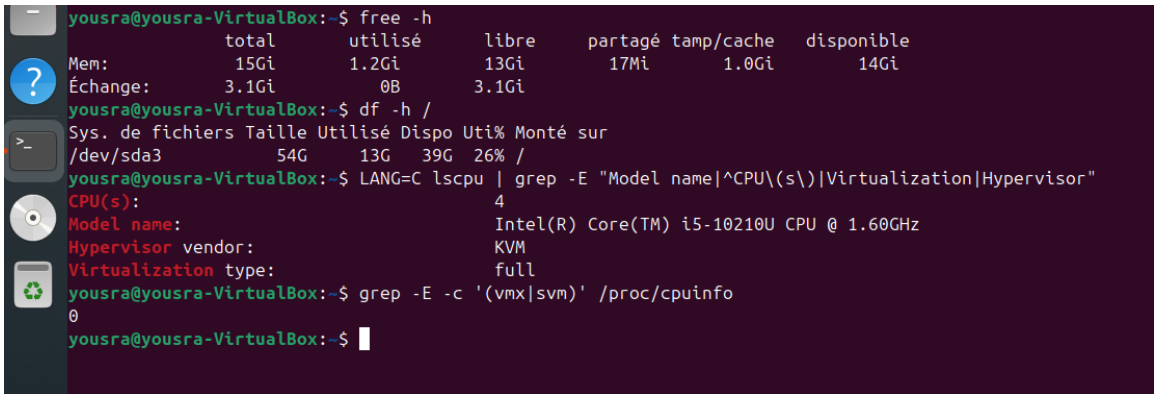


Figure 4: Characteristics of the Ubuntu VM collected from the command line

Hardware virtualization support

In the VM, the command `grep -E -c '(vmx|svm)' /proc/cpuinfo` returns the value **0**: the processor's hardware virtualization extensions are not exposed to the virtual machine. Nested virtualization is therefore not available in our configuration.

Consequence: the KVM hypervisor cannot accelerate the OpenStack instances. Nova uses QEMU in *software emulation* mode (parameter `LIBVIRT_TYPE=qemu`), which is confirmed by the "Hypervisor type" column in Horizon (QEMU). The instances work, but more slowly. We therefore use a lightweight image (CirrOS) and a minimal instance type (*flavor*).

The same check on `compute2` also gives the value 0 and shows that the file `/dev/kvm` does not exist: `compute2` also runs in QEMU emulation.

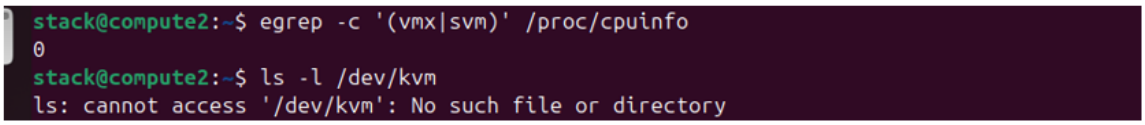


Figure 5: No hardware virtualization on `compute2`

1.5 Choice of the Deployment Method

Methods considered

Method	**Principle**	**Assessment against our constraints**
DevStack	Installation script from source, driven by a `local.conf` file	Quick to set up, suited to testing and learning, not intended for production
Kolla-Ansible	Deployment of OpenStack in containers with Ansible	Production-oriented: heavier preparation than the workshop requires
Packstack	Automated installation for the Red Hat family (RDO)	Different distribution family from the Ubuntu used by the team
Packaged distribution for Ubuntu (MicroStack)	Deployment guided by dedicated tools	Less close to the detailed service configuration that the workshop asks us to document
Manual installation	Installing and configuring each service one by one	Very instructive, but too long for the time available

Table 3: Comparison of the deployment methods considered

Method chosen

We chose **DevStack**, on the `stable/2026.1` branch, installed on **Ubuntu 24.04**, with a **multi-node architecture**: one *controller* node and two *compute* nodes, each being a VirtualBox virtual machine on the PC of a team member [2].

Justification of the choice

- **Suited to a test environment:** DevStack installs a working OpenStack cloud from a configuration file, with the services needed for the practical part of the statement: Keystone, Nova, Neutron, Glance, Placement and Horizon [1]. Cinder, which the statement asks us to study but which the practical part does not need, is not deployed.
- **Consistent version:** we use the stable branch of version 2026.1 on Ubuntu 24.04, the combination we selected based on the OpenStack documentation.
- **Architecture close to a real cloud:** separating the controller (management services) from the compute nodes (instance hosting) reproduces the organization of an enterprise cloud and helps understand the role of each node.
- **Division of work:** with three machines available, each team member installs and administers one node, which meets the requirement that everyone take part in the installation.
- **Sufficient resources:** our machines match the order of magnitude of memory recommended by the official documentation (Table 4).

Fit with the available resources

Role	Documentation recommendation	Our VM	Remark
Controller	1 to 2 CPUs, 8 GB of RAM, 100 GB of disk	About 10 GB of RAM, 4 vCPUs and 60 GB of disk	Meets the memory recommendation
Compute	2 to 4 CPUs or more, 8 GB of RAM or more, 100 GB of disk or more	`compute1`: 4 vCPUs, 16 GB of RAM, 55 GB of disk; `compute2`: about 9.7 GB of RAM, 6 vCPUs and 48 GB of disk	Reduced disk: very small CirrOS instances

Table 4: Hardware requirements of the official documentation and resources of our VMs

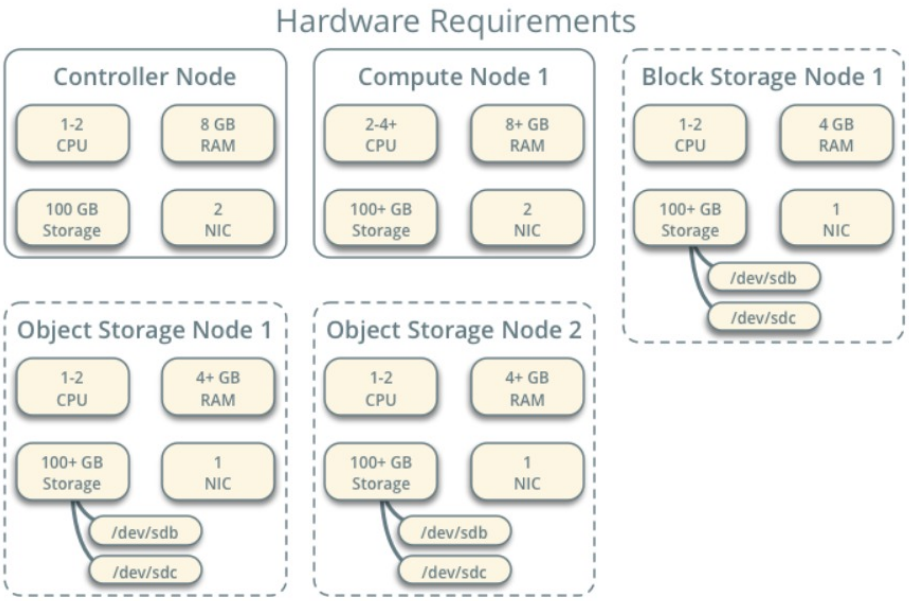


Figure 6: Hardware requirements given in the official OpenStack documentation

The documentation values correspond to a full installation. In our case the disk is smaller, because we only create small CirrOS instances, and the Cinder volume service is not deployed.

Deployed architecture

Node	Role	Owner	Address (VPN)
controller	Management services, database, message queue	Oumaima El Ghalbouni	100.104.158.77
compute1	Instance hosting	Yousra Khallou	100.85.196.18
compute2	Instance hosting	Firdawss El Haddouchi	100.97.155.1

Table 5: Distribution of the nodes within the team

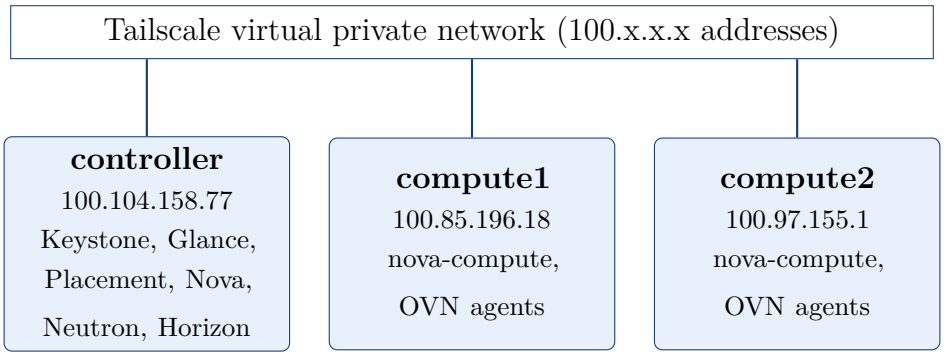


Figure 7: Deployed multi-node architecture: one controller and two compute nodes linked by a VPN

Limits and constraints

- **Software emulation:** without hardware virtualization in the VMs, the instances are slower (QEMU instead of KVM).
- **Network between nodes:** since the three machines are on different networks, they are linked by a VPN (Tailscale), whose interface limits the packet size (MTU) to 1280 bytes, which required a Neutron adjustment (section 2.6).
- **Limited resources:** the 55 GB disk of our VMs is below the official recommendation; we therefore limit the size of images and instances.

2 Part 2 – Installing OpenStack

This part follows the actual order of our work: preparing the three virtual machines (controller, compute1, compute2), the first networking approach and its limits, the chosen solution (the Tailscale virtual private network), then the installation of DevStack on the controller and on the compute nodes, and verification of the cloud.

2.1 Preparing the Controller Node

The controller node, administered by Oumaima El Ghalbouni, hosts the OpenStack management services. It is prepared first, because the compute nodes must be able to reach it. Its virtual machine has about 10 GB of memory and runs Ubuntu 24.04.5 LTS (Figure 9). The preparation follows the same steps as for the other nodes:

1. **Machine name**: the VM is named `controller`.
2. **Clock and firewall**: the clock is synchronized (`System clock synchronized: yes`) and the Ubuntu firewall is inactive.
3. **Dedicated `stack` user**: creation of the user, passwordless `sudo` rights and access to the `/opt/stack` folder.
4. **Fetching DevStack** on the `stable/2026.1` branch.

```
sudo hostnamectl set-hostname controller
timedatectl
sudo ufw status
sudo useradd -s /bin/bash -d /opt/stack -m stack
sudo chmod +x /opt/stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo -u stack -i
git clone https://opendev.org/openstack/devstack -b stable/2026.1
```

```
vboxuser@ubuntu:~/Desktop$ curl -sI https://opendev.org | head -1       # expect 200
HTTP/1.1 200 OK
vboxuser@ubuntu:~/Desktop$ sudo hostnamectl set-hostname controller
sudo nano /etc/hosts
[sudo] password for vboxuser:
vboxuser@ubuntu:~/Desktop$ sudo hostnamectl set-hostname controller
vboxuser@ubuntu:~/Desktop$ timedatectl
               Local time: Sun 2026-11-01 06:21:40 UTC
           Universal time: Sun 2026-11-01 06:21:40 UTC
                 RTC time: Sun 2026-11-01 06:21:40
                Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
vboxuser@ubuntu:~/Desktop$ sudo ufw status
Status: inactive
vboxuser@ubuntu:~/Desktop$ sudo useradd -s /bin/bash -d /opt/stack -m stack
sudo chmod +x /opt/stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo -u stack -i
stack ALL=(ALL) NOPASSWD: ALL
stack@controller:~$ ls -a /opt/stack
```

Figure 8: Machine name, clock, firewall and `stack` user of the controller

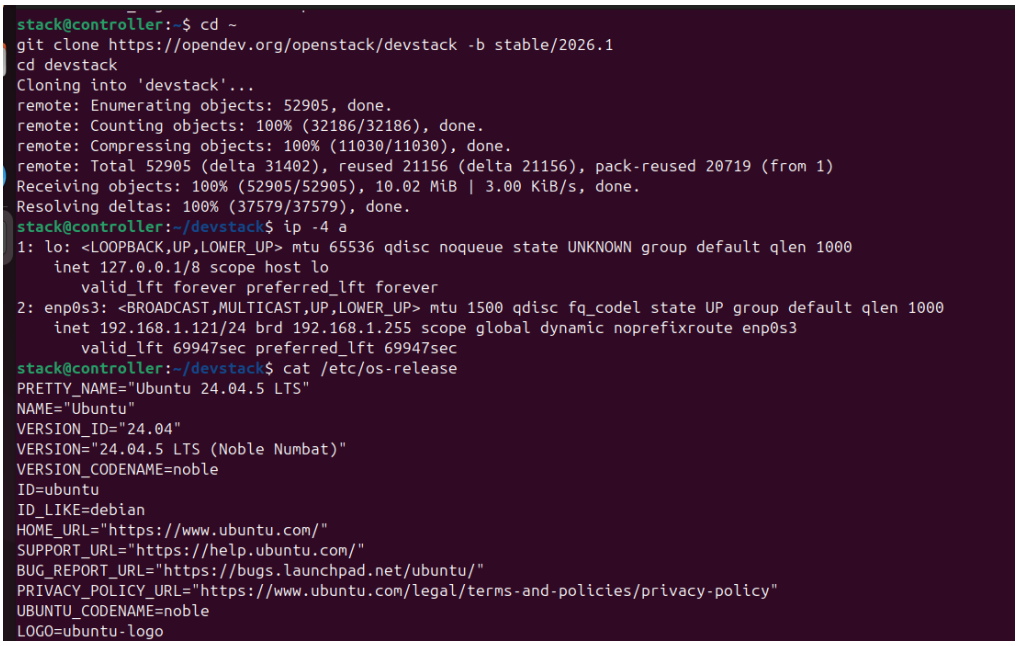


Figure 9: Fetching DevStack and system version of the controller

2.2 Preparing the compute1 Node

The `compute1` VM, administered by Yousra KHALLOU, is the second machine prepared. It was running Ubuntu 22.04.5 and was first upgraded to Ubuntu 24.04.

1. **Upgrade from Ubuntu 22.04 to 24.04**: we checked that a new release was offered, started the upgrade, then verified the version.

```
sudo do-release-upgrade -c
sudo do-release-upgrade
lsb_release -a
```

2. **System update and git installation**:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y git
```

3. **Hostname and time**: a unique name per node (`compute1`), and a check of clock synchronization.

```
sudo hostnamectl set-hostname compute1
timedatectl
```

4. **Dedicated `stack` user**: DevStack must be run by a non-administrator user with passwordless `sudo` rights.

```
sudo useradd -s /bin/bash -d /opt/stack -m stack
```

```
sudo chmod +x /opt/stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo -u stack -i
```

5. **Fetching DevStack**:

```
git clone https://opendev.org/openstack/devstack -b stable/2026.1
```

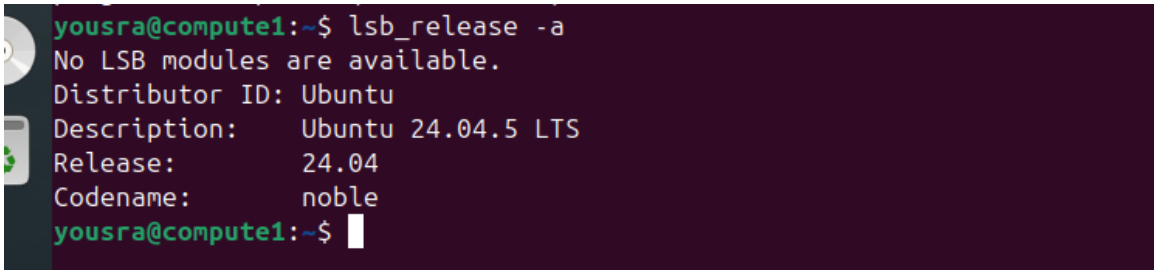


Figure 10: System version of `compute1` after the upgrade (Ubuntu 24.04.5 LTS)

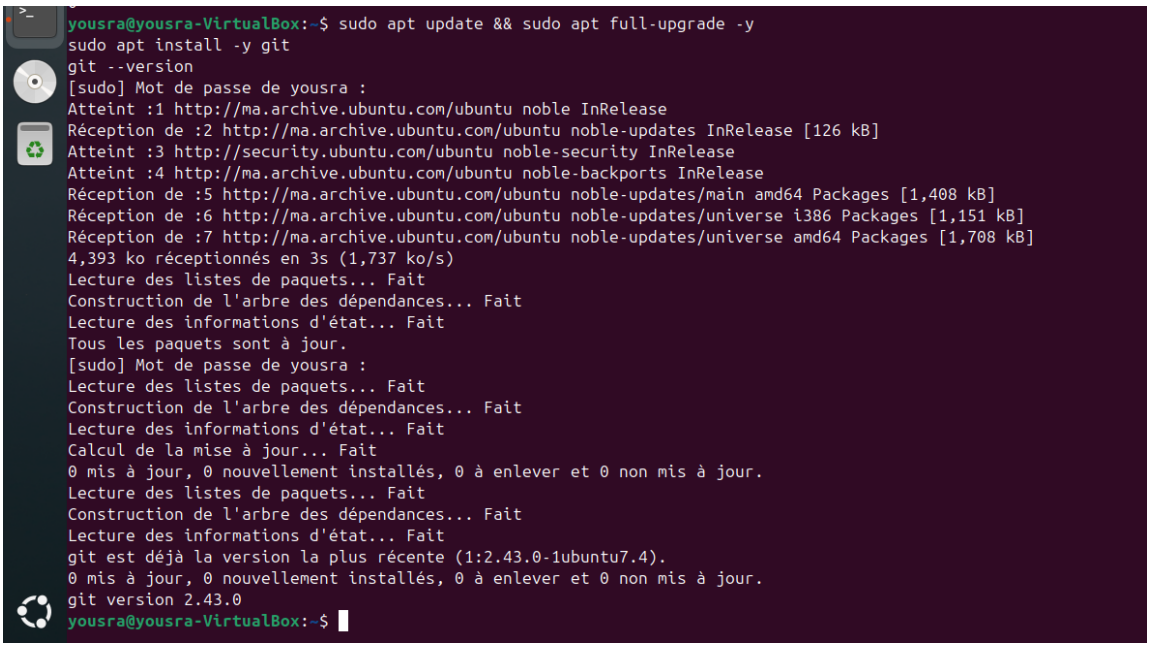


Figure 11: System update and git installation on `compute1`

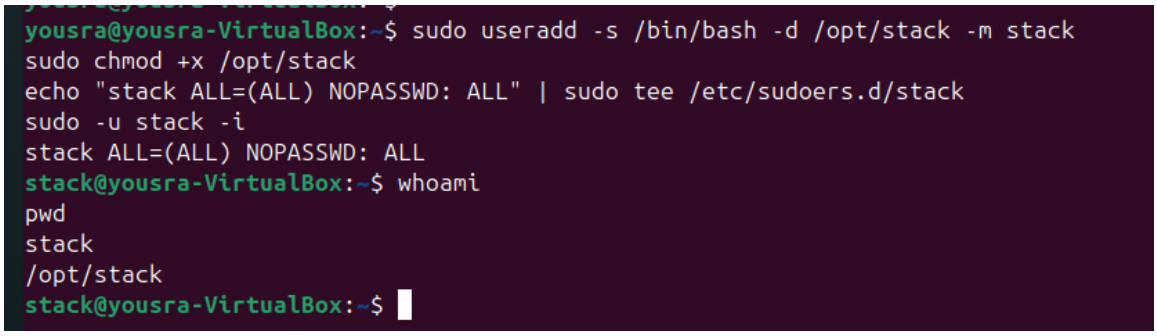


Figure 12: Creation of the `stack` user on `compute1`



Figure 13: Fetching DevStack on `compute1` (branch `stable/2026.1`)

2.3 Preparing the compute2 node

The `compute2` VM, administered by Firdawss El Haddouchi, was prepared following the same steps as `compute1`. It runs Ubuntu 24.04.5 LTS (figure 14).

1. **Hostname**: the VM is renamed `compute2`.
2. **Dedicated `stack` user**: creation of the user, passwordless `sudo` rights, `sudoers` file protected as read-only (`0440`), and access to the `/opt/stack` folder.
3. **git installation** (version 2.43.0).
4. **Fetching DevStack** on the `stable/2026.1` branch.
5. **Clock and firewall**: the time synchronization service was inactive; it was enabled, and the Ubuntu firewall remained inactive.

```
sudo hostnamectl set-hostname compute2
sudo useradd -s /bin/bash -d /opt/stack -m stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo chmod 0440 /etc/sudoers.d/stack
sudo apt install -y git
git clone https://opendev.org/openstack/devstack -b stable/2026.1
sudo timedatectl set-ntp true
```

```
sudo systemctl restart systemd-timesyncd
sudo ufw status
```



Figure 14: System version of `compute2` (Ubuntu 24.04.5 LTS)

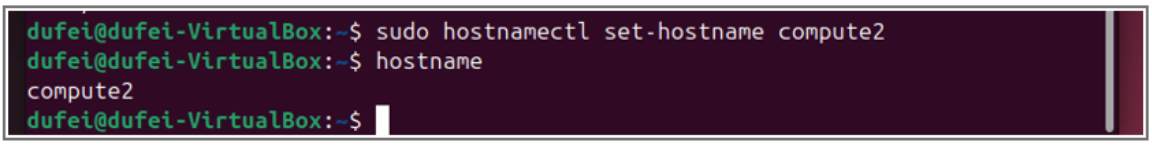


Figure 15: Hostname of `compute2`

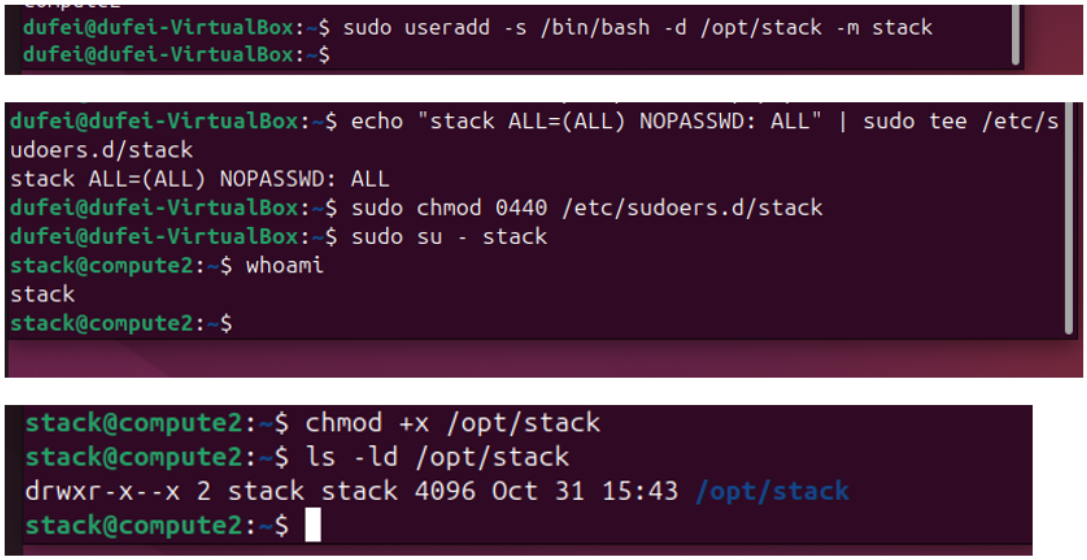


Figure 16: `stack` user on `compute2`

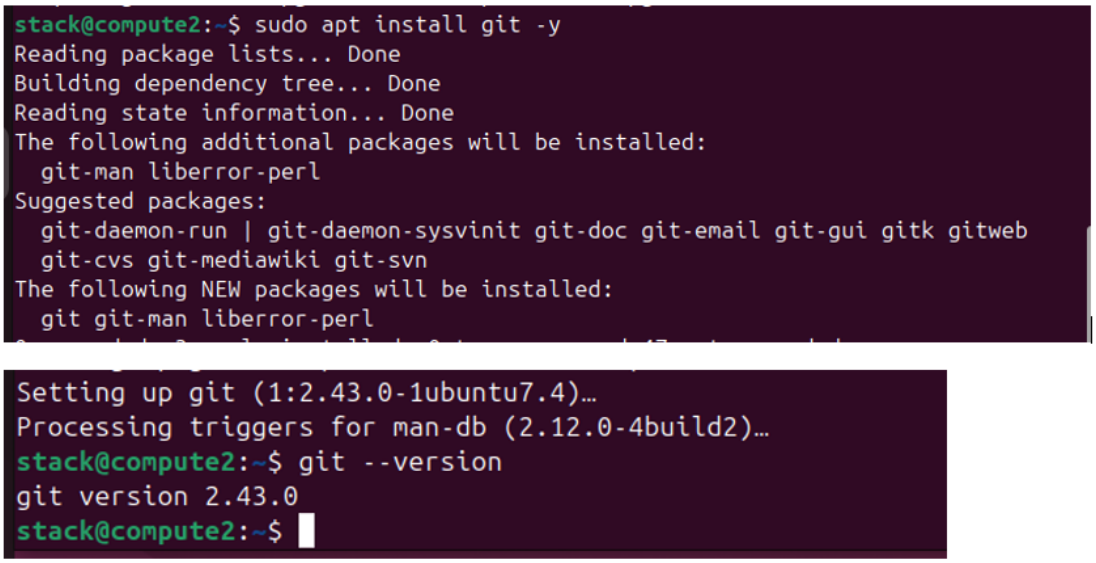


Figure 17: git installation on `compute2`

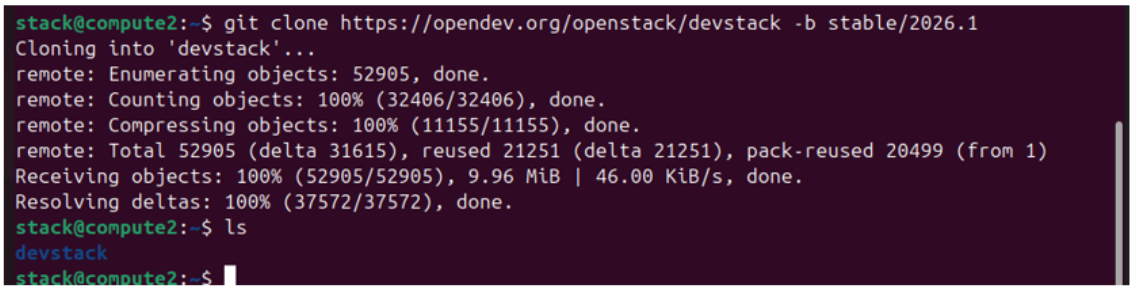


Figure 18: Fetching DevStack on `compute2`

2.4 First networking method: Wi-Fi in bridged mode

To connect the three machines, we first used Wi-Fi: the VirtualBox network adapter of each VM is set to “Bridged Adapter” mode, with a fixed IP address in the `192.168.0.0/24` subnet (controller: `.101`, compute1: `.102`, compute2: `.103`) and an identical `/etc/hosts` file on each machine. Connectivity tests between the VMs succeeded with no packet loss.

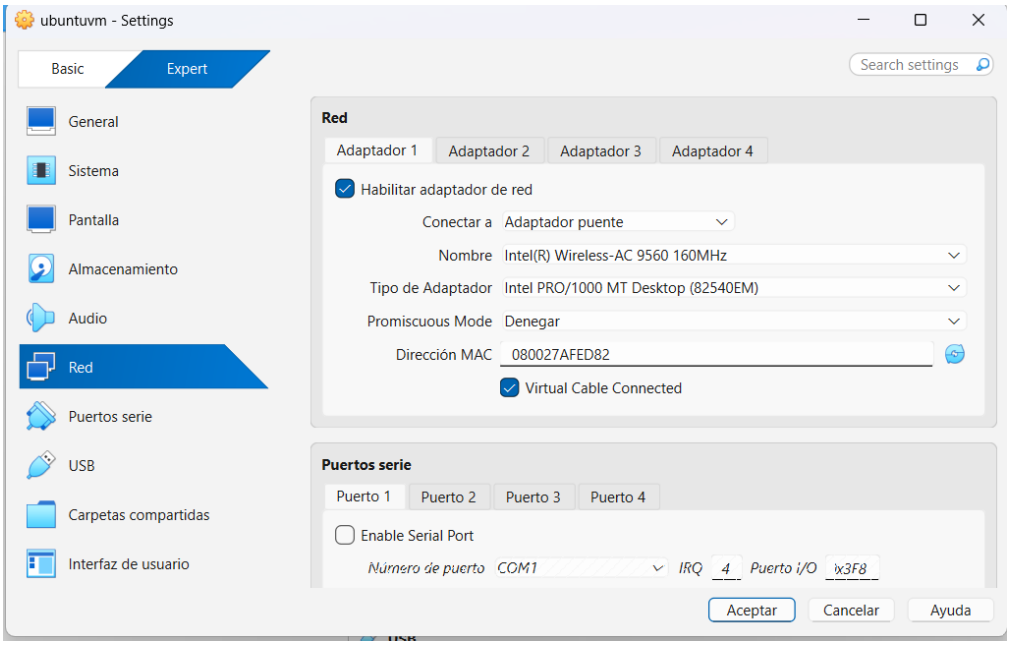


Figure 19: VirtualBox network adapter of the controller in “Bridged Adapter” mode

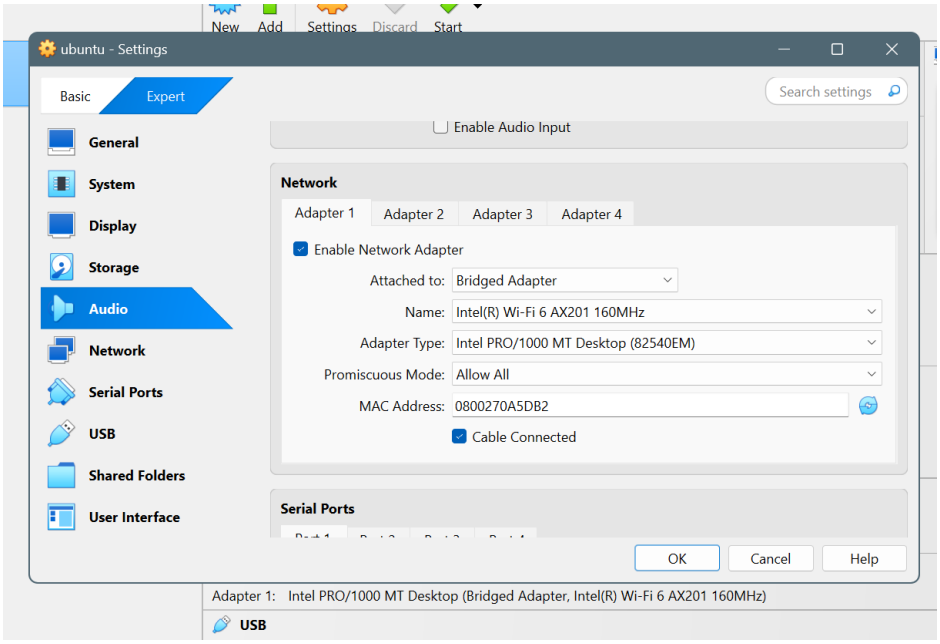


Figure 20: VirtualBox network adapter of `compute1` in “Bridged Adapter” mode

```
stack@controller:~/devstack$ sudo nmcli con mod "netplan-enp0s3" ipv4.method manual \
  ipv4.addresses 192.168.0.101/24 \
  ipv4.gateway 192.168.0.1 \
  ipv4.dns "192.168.0.1 8.8.8.8"
sudo nmcli con up "netplan-enp0s3"
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/4)
```

Figure 21: Fixed IP address of the controller

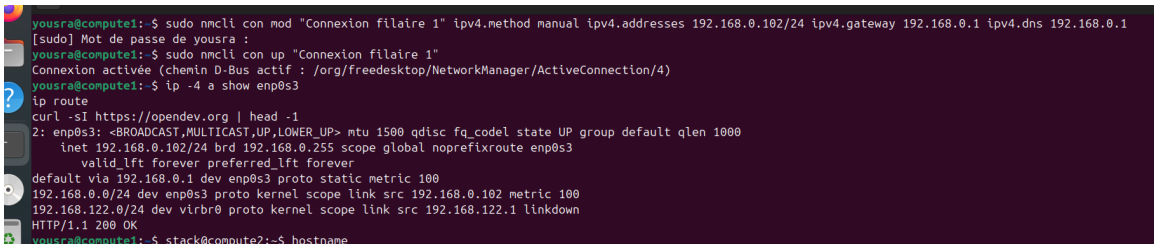


Figure 22: Fixed IP address of `compute1` and Internet access

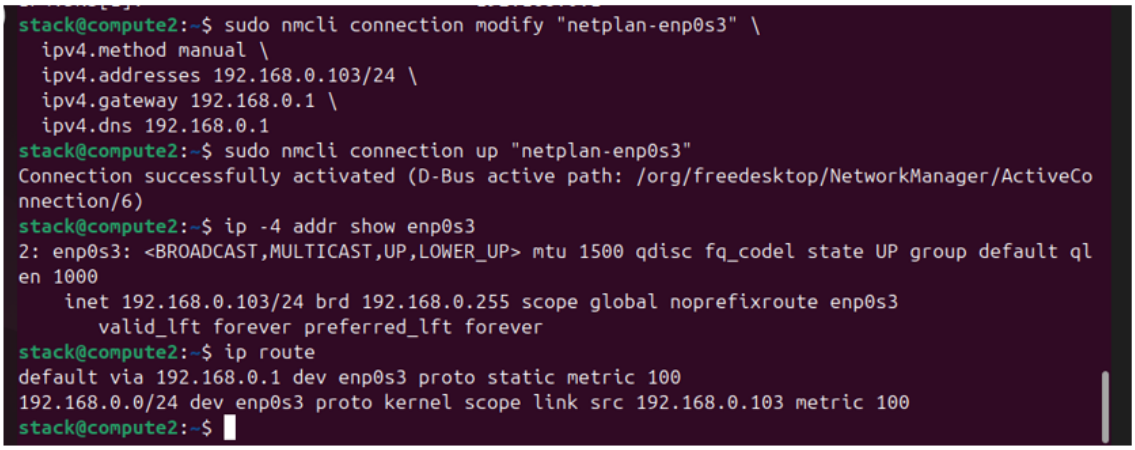


Figure 23: Fixed IP address of `compute2`

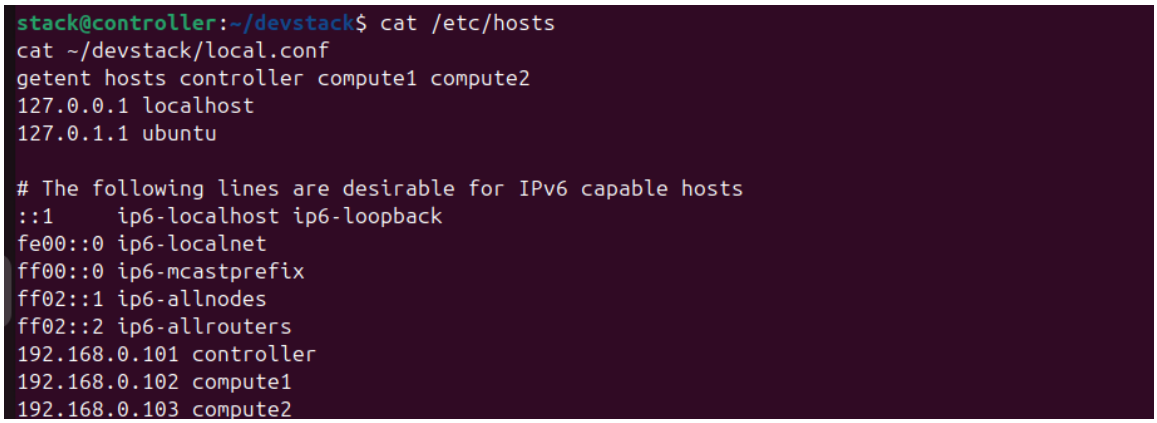


Figure 24: `/etc/hosts` file of the controller

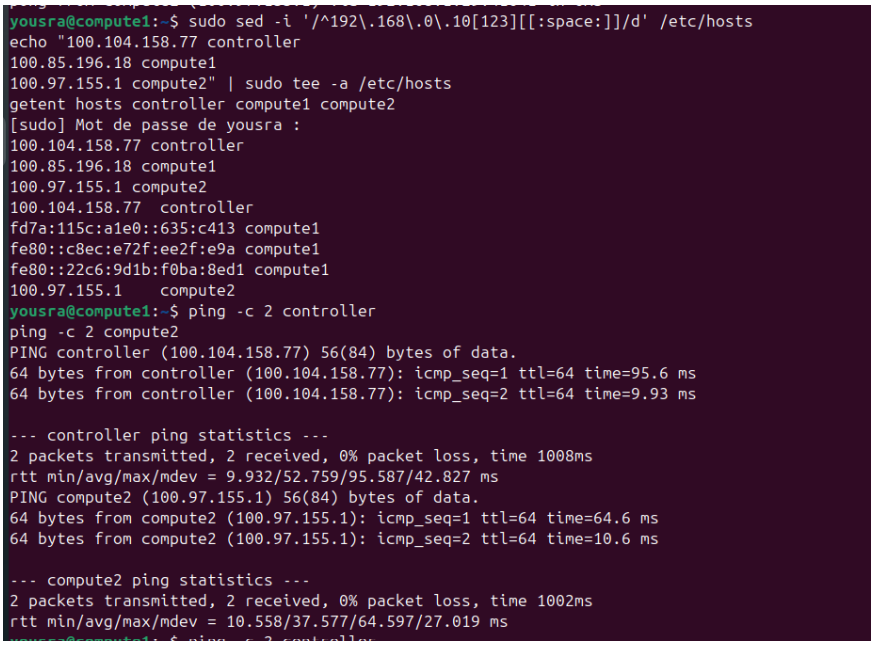


Figure 25: `/etc/hosts` file of `compute1`

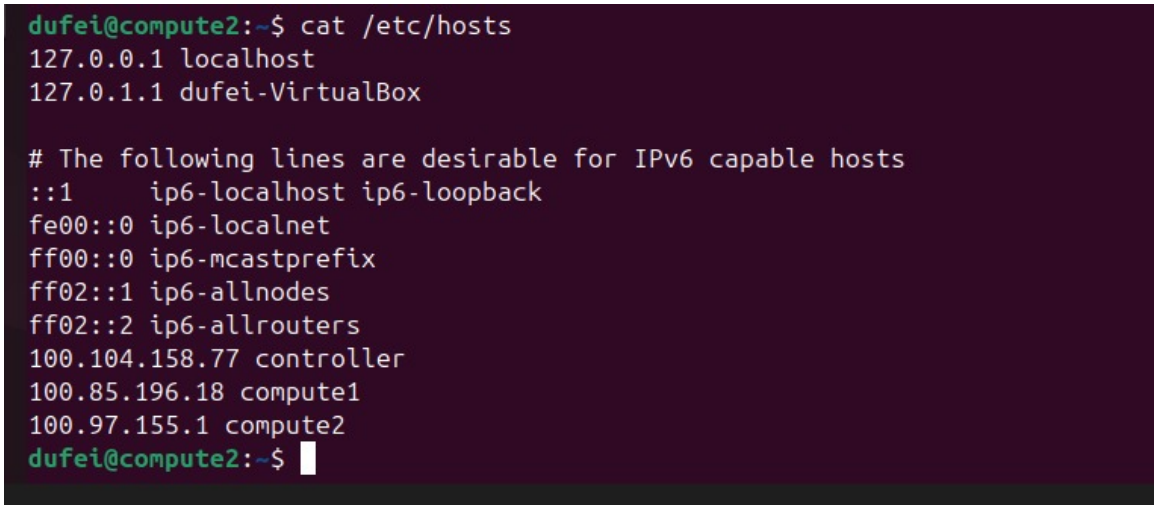


Figure 26: `/etc/hosts` file of `compute2`

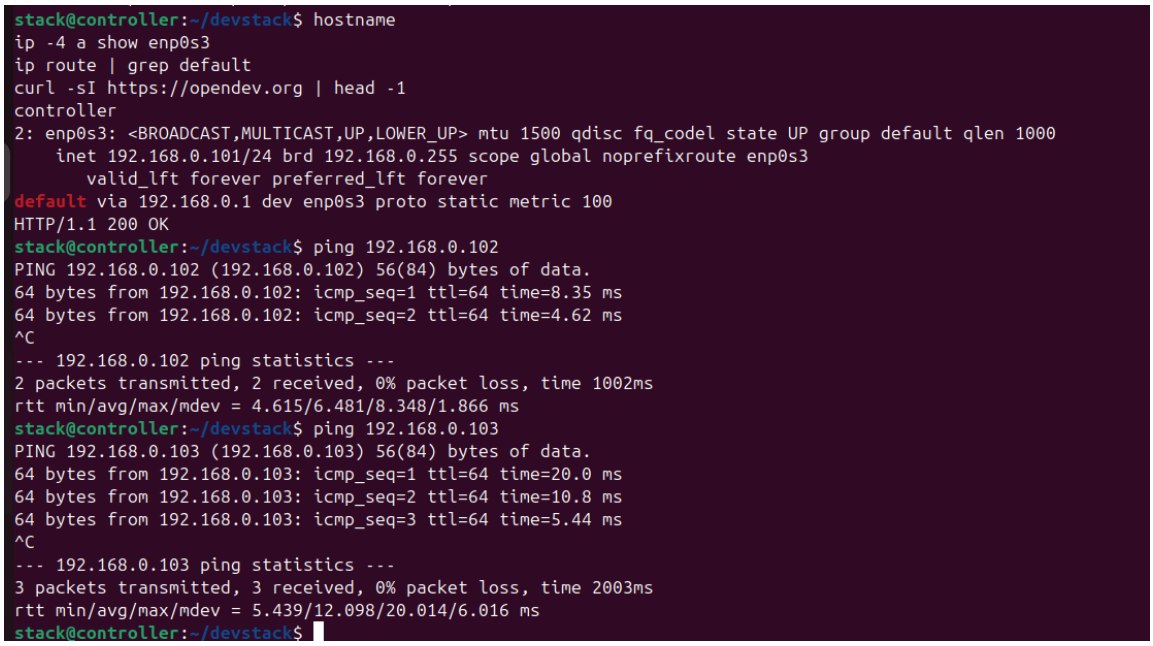


Figure 27: Connectivity tests from the controller to `compute1` and `compute2`

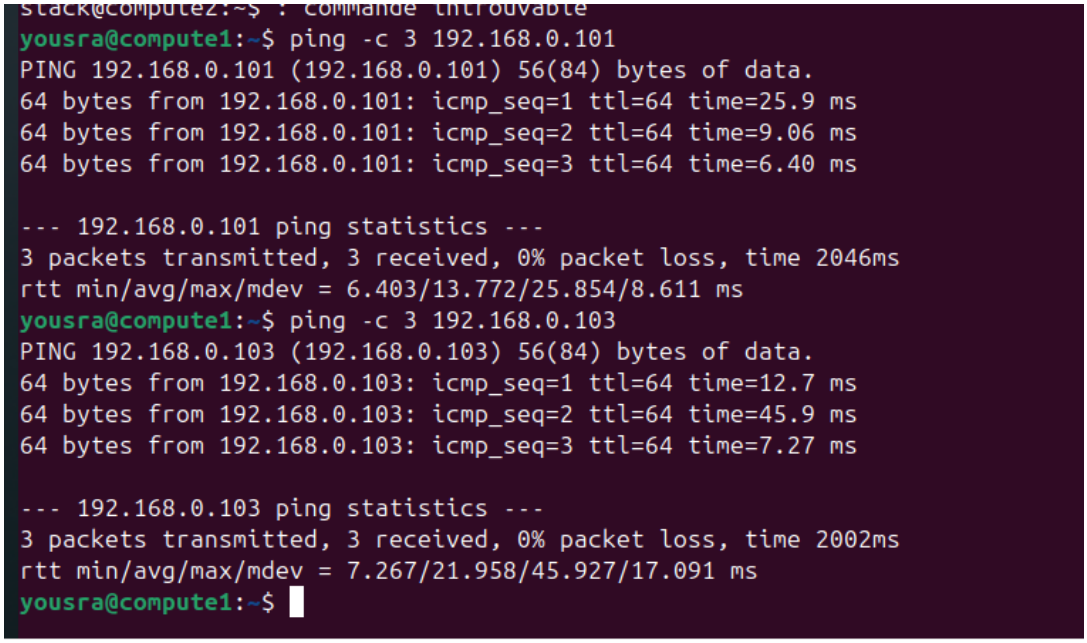


Figure 28: Connectivity tests from `compute1` to the controller and compute2

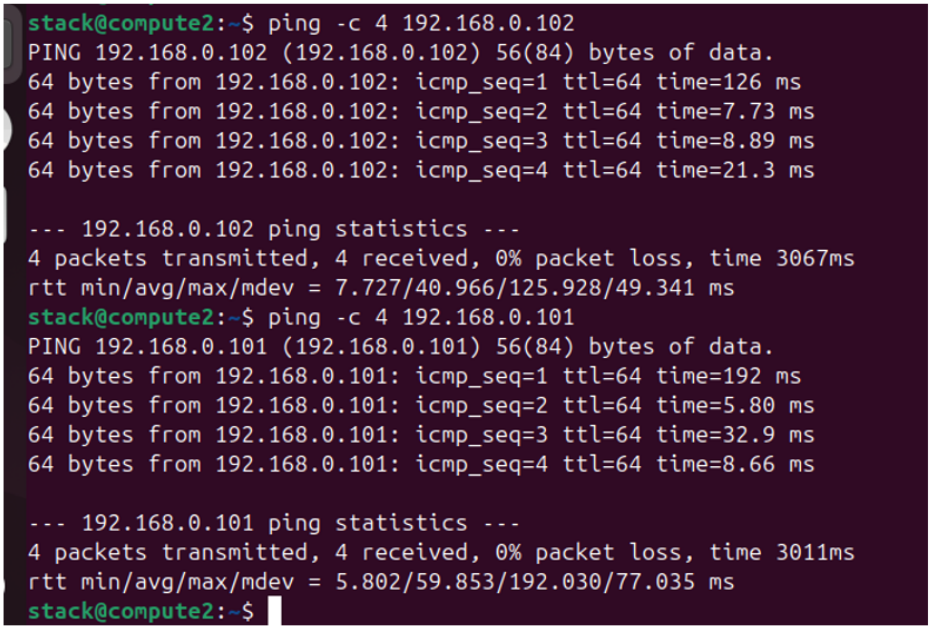


Figure 29: Connectivity tests from `compute2` to `compute1` and the controller

2.5 Limitations of the first method

This first method showed several limitations:

- **Insufficient bandwidth:** during the first run of `./stack.sh` on the controller, the download of `pip` failed with a timeout error (`ReadTimeoutError`), the measured throughput being about 8 kB/s. We lengthened the `pip` timeouts, then switched Wi-Fi networks.
- **Blocked ping:** on one of the Wi-Fi networks tested, ICMP traffic was blocked, even towards the gateway. `ping` could no longer be used to test connectivity; we used web connections (`curl`) instead.
- **Network-dependent addresses:** the fixed IP addresses, the `/etc/hosts` file and the `local.conf` files were tied to the Wi-Fi subnet. When changing Wi-Fi, they became unusable: the controller thus moved from the `192.168.1.0/24` subnet to the `192.168.0.0/24` subnet (figure 30).
- **Machines on different networks:** the team's PCs were not always on the same local network (for example, `compute2` ended up in the `192.168.11.0/24` subnet).

A network common to the three machines, independent of the Wi-Fi in use, was therefore needed.

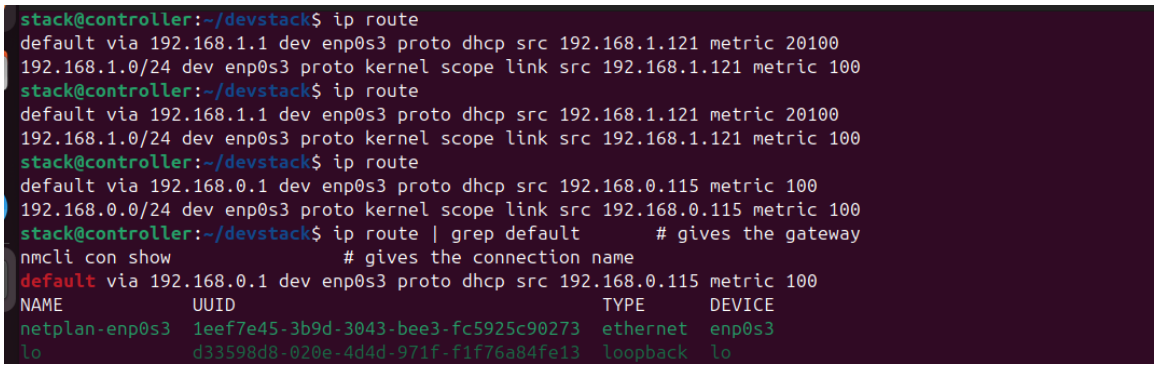


Figure 30: Network change of the controller (subnets `192.168.1.0/24` then `192.168.0.0/24`)

2.6 Solution: the Tailscale virtual private network

We connected the three VMs with a **Tailscale** virtual private network (VPN). It is installed in each Ubuntu VM (not on the Windows PCs) from the official Tailscale repository, then each VM joins the same private network:

```
sudo tailscale up --hostname=compute1
tailscale status
tailscale ping controller
```

(the hostname is `controller`, `compute1` or `compute2` depending on the VM). Each VM then has a stable Tailscale address (table 5), used as the management address by OpenStack. Traffic between the nodes is encrypted by the VPN and no longer depends on Wi-Fi.

The `tailscale ping` tests showed direct connections (without a relay): 35 ms from `compute1` to the controller, 7 ms from `compute2` to `compute1` and 74 ms from `compute2` to the controller; from the controller, 9 ms to `compute1` and 45 ms to `compute2`. At other times, the connection with `compute1` went through a Tailscale relay (figure 34), which works but increases latency. A regular `ping` from `compute2` to the controller's Tailscale address receives 4 replies out of 4, with an average delay of about 160 ms (figure 32). The VPN interface imposes an MTU of 1280 bytes, hence a Neutron setting on the controller (next section).

The `/etc/hosts` file of each VM is then updated: the old Wi-Fi-related lines are removed and replaced by the Tailscale addresses.

```
sudo sed -i '/^\(192\.168\|100\.\)[0-9.]*[[:space:]]\+\(controller\|compute1\|
    compute2\)$/d' /etc/hosts
echo "100.104.158.77 controller
100.85.196.18 compute1
100.97.155.1 compute2" | sudo tee -a /etc/hosts
getent hosts controller compute1 compute2
```

Adding the Tailscale repository caused a signature error during `apt update` (`NO_PUBKEY`), which stopped the DevStack installation. It is fixed by downloading the repository's official key again:

```
curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/noble.noarmor.gpg | \
  sudo tee /usr/share/keyrings/tailscale-archive-keyring.gpg >/dev/null
sudo apt update
```

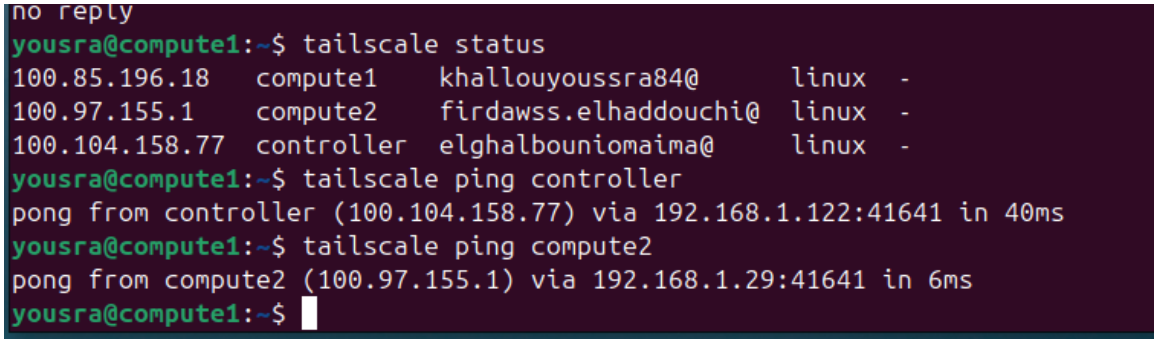


Figure 31: Tailscale VPN status and connectivity tests between the nodes (view from `compute1`)

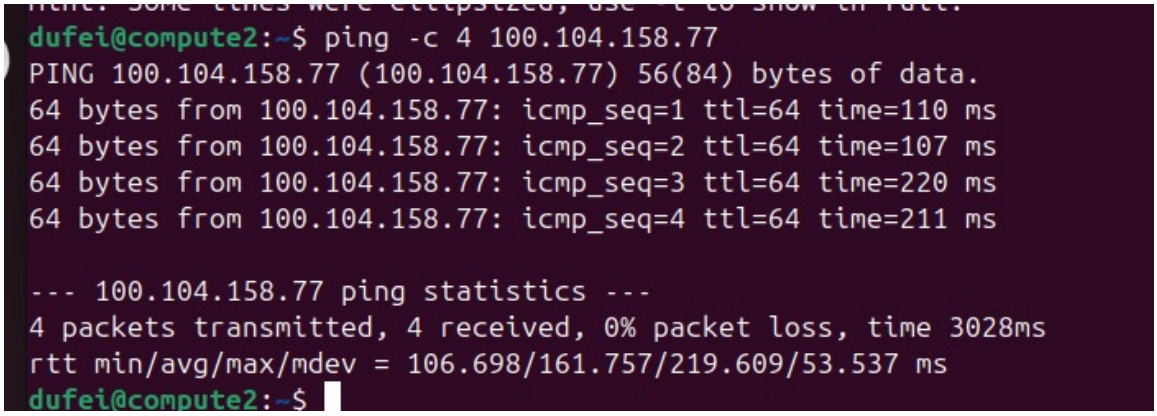


Figure 32: Connectivity test from `compute2` to the controller through the VPN

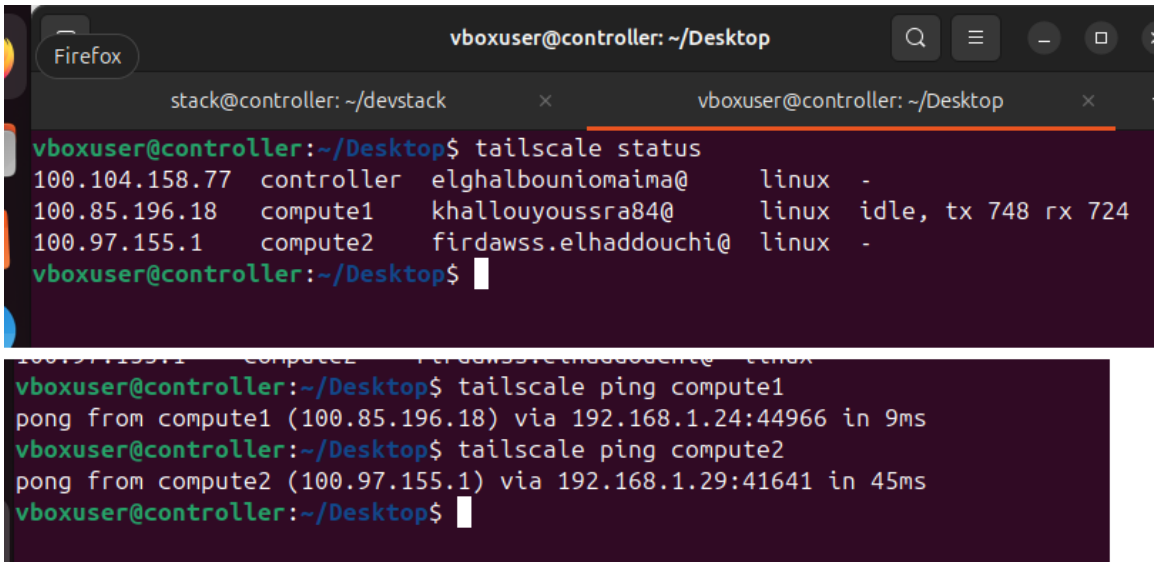


Figure 33: Tailscale VPN status and connectivity tests seen from the controller



Figure 34: VPN status seen from the controller: connection to `compute1` through a relay, direct connection to `compute2`

2.7 Installing the controller node

The controller is installed first, because the compute nodes must be able to reach its database, its message queue and its APIs. Its `local.conf` file is as follows (passwords hidden):

```
[[local|localrc]]
GIT_DEPTH=1
GIT_TIMEOUT=600
HOST_IP=100.104.158.77
SERVICE_HOST=100.104.158.77
MULTI_HOST=1
ADMIN_PASSWORD=<lab password>
DATABASE_PASSWORD=<lab password>
```

```
RABBIT_PASSWORD=<lab password>
SERVICE_PASSWORD=<lab password>
LOGFILE=$DEST/logs/stack.sh.log
LOG_COLOR=False
disable_service n-cpu

[[post-config|/etc/neutron/neutron.conf]]
[DEFAULT]
global_physnet_mtu = 1280
```

The key points of this configuration are the following:

- `HOST_IP` and `SERVICE_HOST` are the controller's Tailscale address;
- `disable_service n-cpu` prevents the controller from hosting instances: this task is reserved for the compute nodes;
- `GIT_DEPTH=1` and `GIT_TIMEOUT=600` limit the amount of code downloaded and abort a download that exceeds ten minutes;
- the four passwords must be identical on the three nodes;
- the `global_physnet_mtu = 1280` setting adapts the packet size of the OpenStack networks to that of the VPN: the private networks created afterwards have an MTU of 1222 bytes (section 3.3).

The installation is started with the `./stack.sh` script, as the `stack` user, from the `devstack` folder. It required several attempts: the first run failed because of the Wi-Fi throughput (section 2.5); after switching to Tailscale, the download of the Keystone repository stayed frozen for more than an hour, which led us to add `GIT_DEPTH` and `GIT_TIMEOUT`; finally, the passwords in the `local.conf` file were corrected and made uniform (table 6, difficulties 8 and 9).

2.8 Installing the compute nodes

A compute node installs only the services needed to host instances: `nova-compute`, the OVN metadata agent and the Open vSwitch / OVN components. It connects to the controller's services through `SERVICE_HOST`. The `local.conf` file of `compute1` is as follows:

```
[[local|localrc]]
GIT_DEPTH=1
GIT_TIMEOUT=600
HOST_IP=100.85.196.18
SERVICE_HOST=100.104.158.77
ADMIN_PASSWORD=<lab password>
DATABASE_PASSWORD=<lab password>
RABBIT_PASSWORD=<lab password>
```

```
SERVICE_PASSWORD=<lab password>
LOGFILE=$DEST/logs/stack.sh.log
LOG_COLOR=False
DATABASE_TYPE=mysql
MYSQL_HOST=$SERVICE_HOST
RABBIT_HOST=$SERVICE_HOST
ENABLED_SERVICES=n-cpu,placement-client,ovn-controller,ovs-vswitchd,ovsdb-server,q-
    ovn-metadata-agent
NOVA_VNC_ENABLED=True
NOVNCPROXY_URL="http://$SERVICE_HOST:6080/vnc_lite.html"
VNCSERVER_LISTEN=$HOST_IP
VNCSERVER_PROXYCLIENT_ADDRESS=$VNCSERVER_LISTEN
LIBVIRT_TYPE=qemu
```

The parameters `DATABASE_TYPE`, `MYSQL_HOST`, `RABBIT_HOST` and `ENABLED_SERVICES` follow the DevStack multi-node guide [2]. `LIBVIRT_TYPE=qemu` forces software emulation (section 1.4). The Cinder volume service (`c-vol`) is not enabled. The `local.conf` of `compute2` is identical, with `HOST_IP=100.97.155.1` and, in addition, the line `GIT_BASE=https://github.com`, which makes the project code download from GitHub.

The installation of `compute1` completed successfully in 470 seconds; the final summary indicates DevStack version `2026.1` on Ubuntu 24.04. The log contains a warning stating that the OVN metadata agent is deprecated (since version 2025.2) in favor of the OVN agent with metadata extension: this is a warning, not an error, and the service works. The `devstack@n-cpu` service is active. For additional reading on installing a compute node on Ubuntu, we consulted the Nova installation guide [3].



Figure 35: `local.conf` file of the `compute1` node

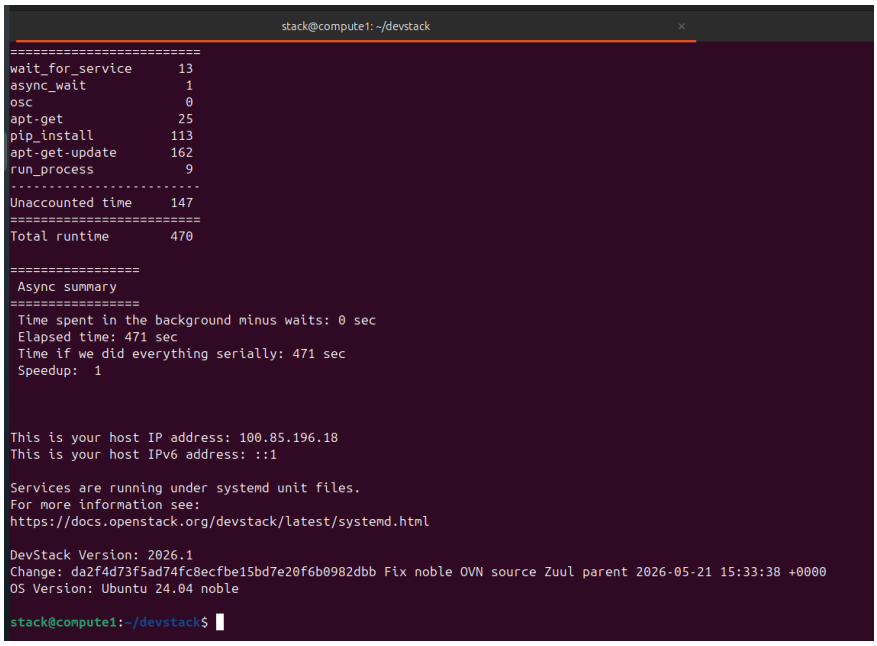


Figure 36: End of the `compute1` installation (`stack.sh` summary)



Figure 37: `devstack@n-cpu` service active on `compute1`

The installation of `compute2` follows the same procedure, with its Tailscale address as `HOST_IP`. Its `devstack@n-cpu` service is active (running since 5 November 2026 at 19:23 UTC according to the screenshot), and the node appears in the `up` state in the service and hypervisor lists of the next section.

```
dufei@compute2:~$ sudo -u stack cat /opt/stack/devstack/local.conf
[[local|localrc]]
GIT_BASE=https://github.com
GIT_DEPTH=1
GIT_TIMEOUT=600
HOST_IP=100.97.155.1
SERVICE_HOST=100.104.158.77
ADMIN_PASSWORD=
DATABASE_PASSWORD=
RABBIT_PASSWORD=
SERVICE_PASSWORD=
LOGFILE=$DEST/logs/stack.sh.log
LOG_COLOR=False
DATABASE_TYPE=mysql
MYSQL_HOST=$SERVICE_HOST
RABBIT_HOST=$SERVICE_HOST
ENABLED_SERVICES=n-cpu,placement-client,ovn-controller,ovs-vswitchd,ovsdb-server,q-ovn-metadat
a-agent
NOVA_VNC_ENABLED=True
NOVNCPROXY_URL="http://$SERVICE_HOST:6080/vnc_lite.html"
VNCSERVER_LISTEN=$HOST_IP
VNCSERVER_PROXYCLIENT_ADDRESS=$VNCSERVER_LISTEN
LIBVIRT_TYPE=qemu
```

Figure 38: `local.conf` file of the `compute2` node

```
pip_install          709
apt-get-update         3
run_process            9
git_timed            450
-------------------------
Unaccounted time     129
=========================
Total runtime        2068

=================
 Async summary
=================
 Time spent in the background minus waits: 0 sec
 Elapsed time: 2068 sec
 Time if we did everything serially: 2068 sec
 Speedup:  1


This is your host IP address: 100.97.155.1
This is your host IPv6 address: ::1

Services are running under systemd unit files.
For more information see:
https://docs.openstack.org/devstack/latest/systemd.html

DevStack Version: 2026.1
Change: da2f4d73f5ad74fc8ecfbe15bd7e20f6b0982dbb Fix noble OVN source Zuul parent 2026-05-21 1
5:33:38 +0000
OS Version: Ubuntu 24.04 noble
```

Figure 39: End of the `compute2` installation (`stack.sh` summary)

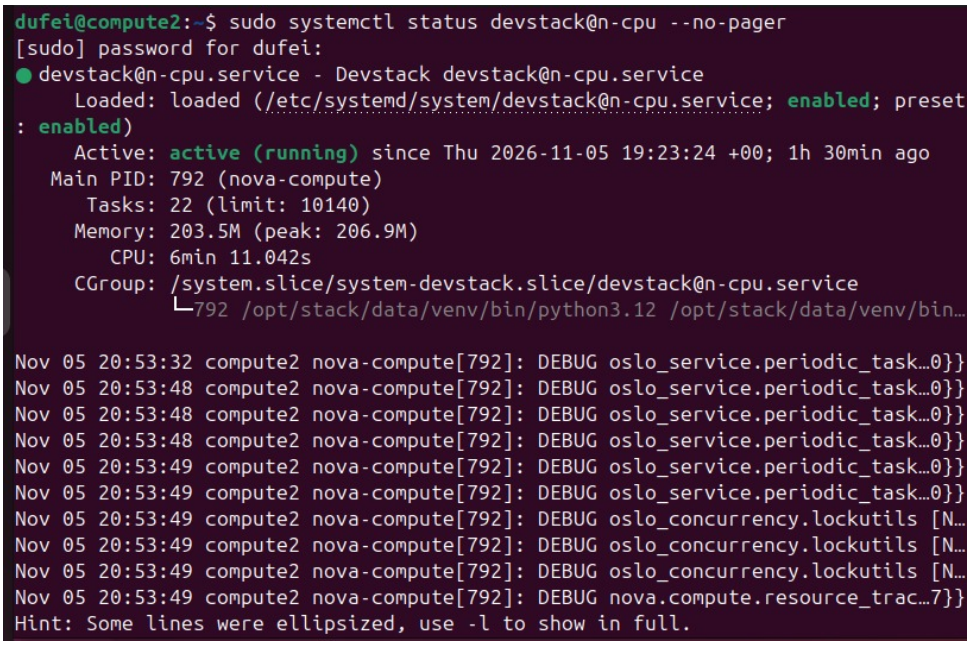


Figure 40: `devstack@n-cpu` service active on `compute2`

2.9 Verifying the services

Once the compute installation is finished, the controller must discover the new node, then the services are listed with the command-line client:

```
cd ~/devstack
./tools/discover_hosts.sh
source openrc admin admin
openstack compute service list
openstack hypervisor list
```

The compute service list shows the controller's management services (`nova-scheduler` and two `nova-conductor`) and the `nova-compute` services of `compute1` and `compute2`, all in the `up` and `enabled` state. This reading of the evening of 5 November 2026 was obtained from `compute2`, which shows that this node does reach the controller's API. The hypervisor list shows `compute1` (address `100.85.196.18`) and `compute2` (address `100.97.155.1`), both of QEMU type. During our tests, the `nova-compute` service of `compute1` went to the `down` state, for lack of updates to the controller, while `compute2` remained `up`, then it came back to the `up` state (section 2.11).

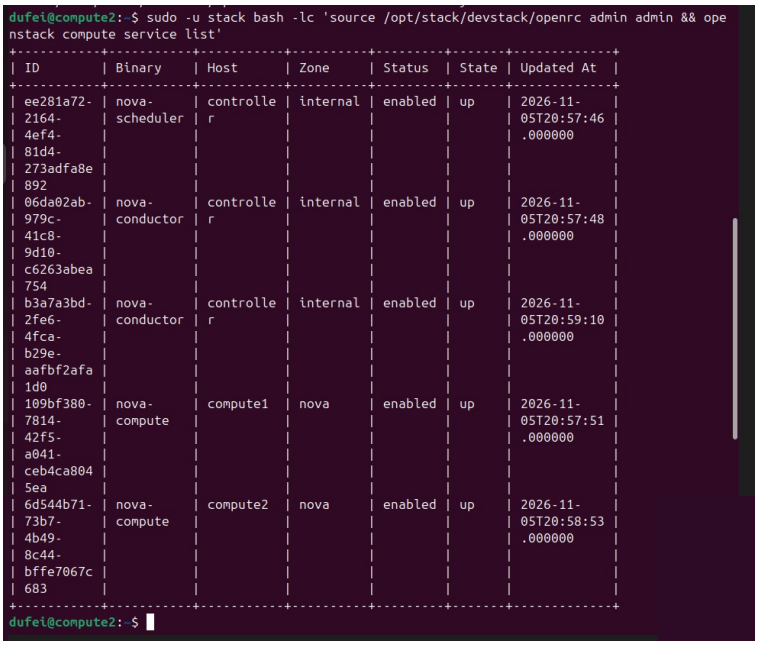


Figure 41: Compute services: controller, `compute1` and `compute2` in the `up` state

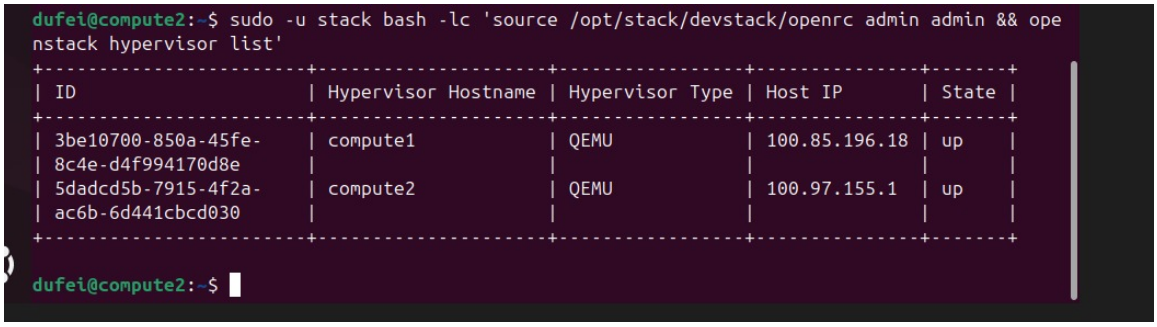


Figure 42: Hypervisor list: `compute1` and `compute2` (QEMU, `up`)

The same information can be viewed in Horizon (*Admin* → *System* → *System Information*). The "Services" tab lists the six services registered in the Keystone catalog: `keystone`, `placement`, `nova`, `neutron`, `nova_legacy` (Nova compatibility entry) and `glance`, all reachable through the controller's address. The Cinder service does not appear, because it is not deployed. The "Compute Services" tab confirms the Nova services, and the "Network Agents" tab shows four active agents: the Neutron OVN agent and the OVN Controller agent of the controller, as well as the OVN Controller agent and the OVN metadata agent of `compute1`.

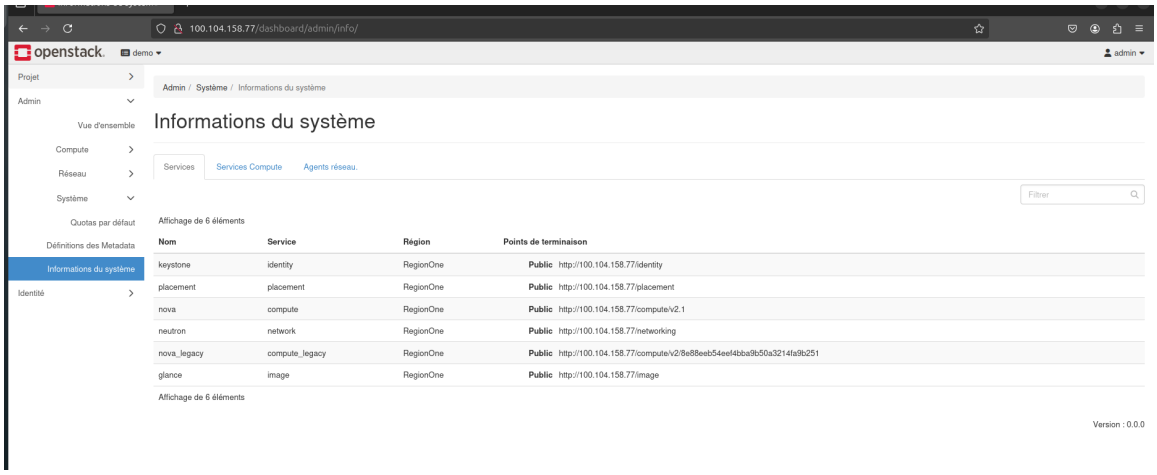


Figure 43: Services registered in the Keystone catalog (Horizon)

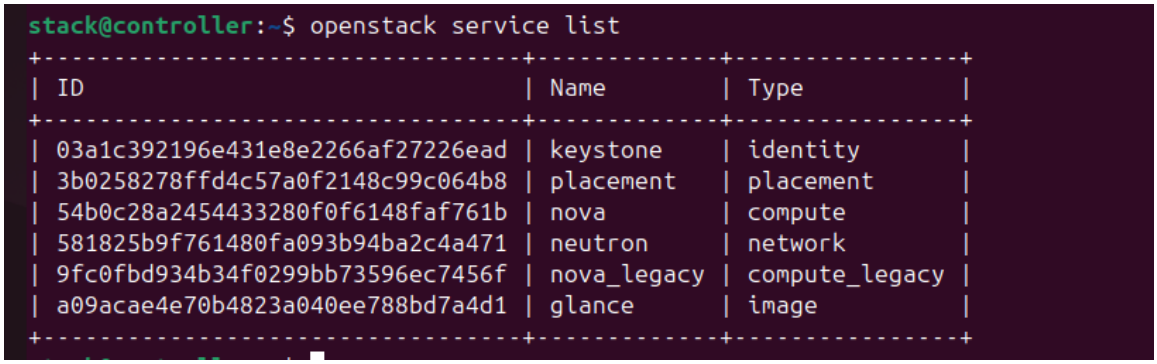


Figure 44: Registered services (command line, `openstack service list`)

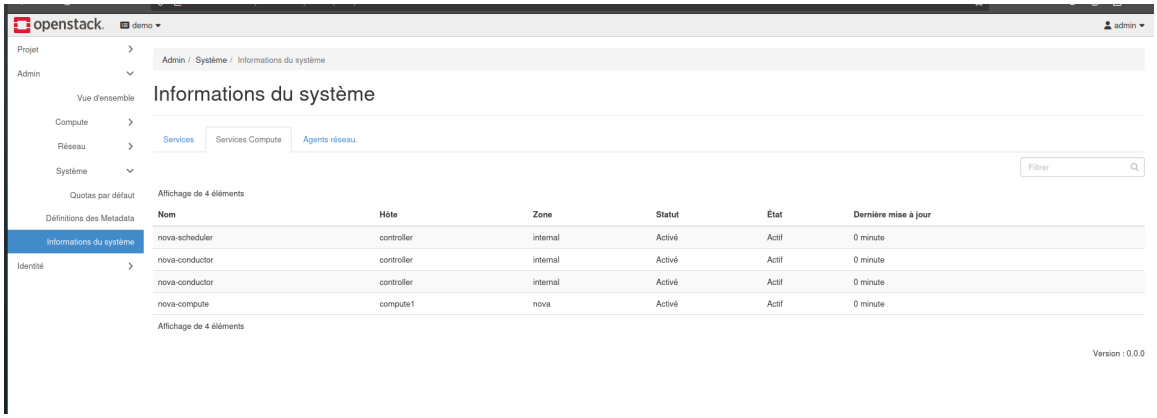


Figure 45: Compute services in Horizon

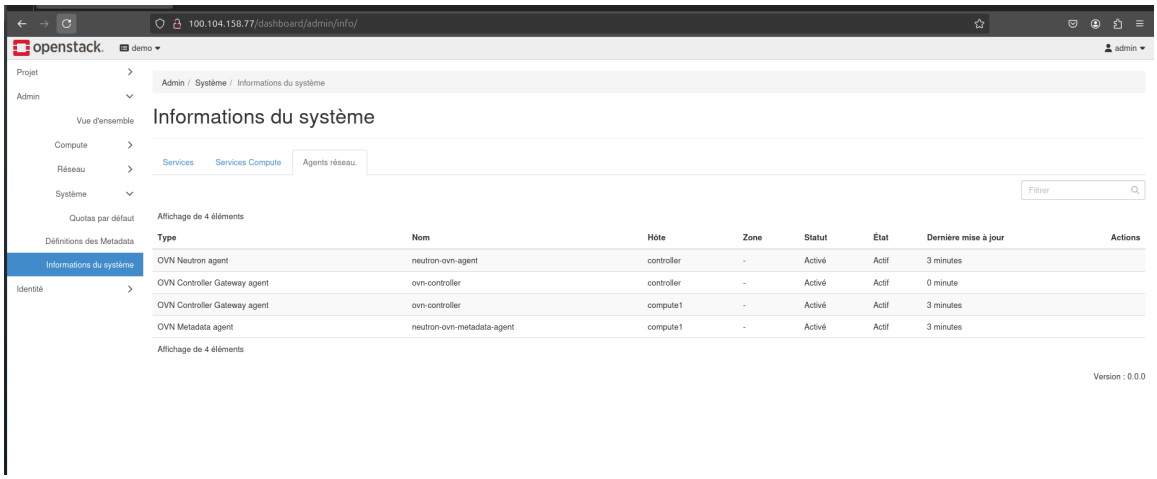


Figure 46: Network agents in Horizon

2.10 Accessing the Horizon administration interface

The Horizon interface is accessible from any VM connected to the VPN, at the address `http://100.104.158.77/dashboard`, with the `admin` user and the password defined in the `local.conf` file (project `demo`). The *Admin* → *Compute* → *Hypervisors* page displays the `compute1` node of QEMU type, with 15.6 GB of memory and 53 GB of local storage.

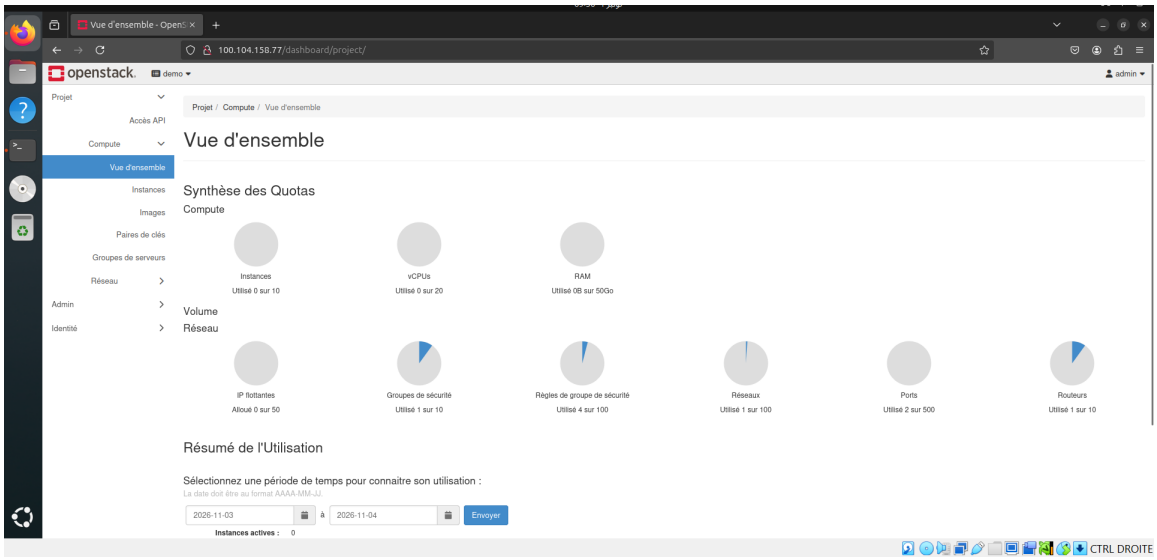


Figure 47: Horizon home page after login

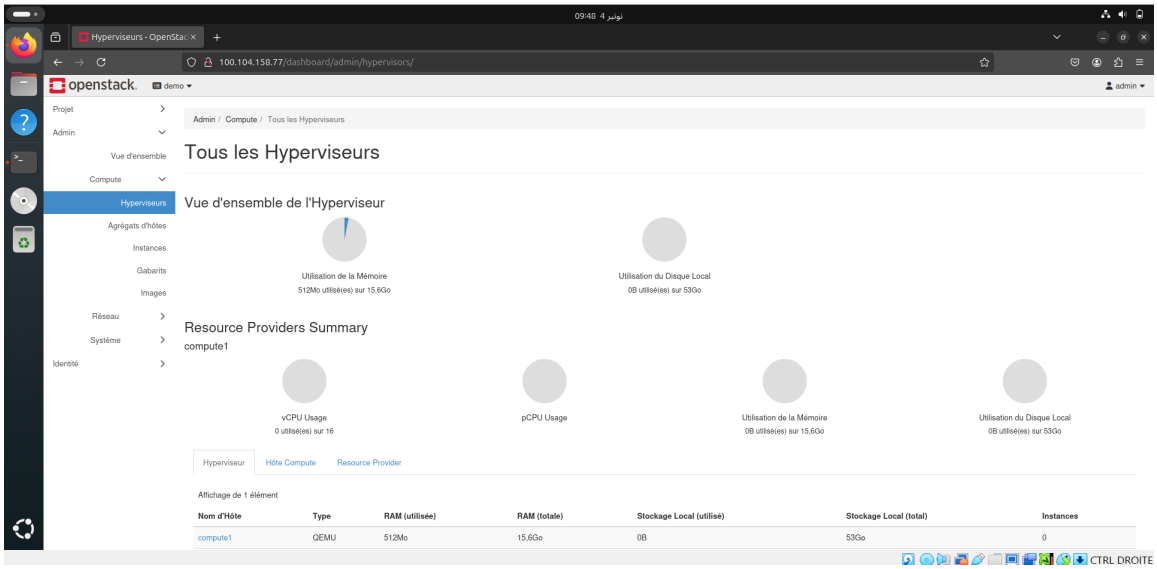


Figure 48: Hypervisors in Horizon: `compute1` node of QEMU type

2.11 Difficulties encountered and solutions adopted

No.	Difficulty	Cause	Solution
1	User created as “Yousrastack” instead of “stack”	Typing error in the `useradd` command	`sudo userdel -r`, then re-creation of the `stack` user
2	`sudo: unknown user stack`	`useradd` command forgotten before `sudo -u stack`	Commands re-run in the right order
3	First `do-release-upgrade` with no new release offered	Cause not identified	Check of `Prompt=lts` then `do-release-upgrade -c`, which offered 24.04.5
4	`ping` fails, even towards the gateway	Wi-Fi blocks ICMP traffic	Test with a real web connection (`curl`)
5	Clock not synchronized (`System clock synchronized: no`): one-hour offset on `compute1`, NTP service inactive on `compute2`	VM clock offset; synchronization disabled	`sudo timedatectl set-ntp true` (`compute2`) and `sudo systemctl restart systemd-timesyncd`
6	Leftovers of an old DevStack installation in `/opt/stack`	Installation attempted before switching to multi-node	`./unstack.sh` then `./clean.sh`, with verification of the cleanup
7	`ReadTimeoutError` while downloading `pip`	Very low Wi-Fi throughput (about 8 kB/s)	Longer timeouts (`PIP_DEFAULT_TIMEOUT`, `PIP_RETRIES`), then change of Wi-Fi and switch to the VPN
8	Keystone repository download frozen for more than an hour	Blocked connection, no abort timeout in DevStack	Stopping the `git clone`, then `GIT_DEPTH=1` and `GIT_TIMEOUT=600`
9	Non-identical passwords and a space after the `=` sign in `DATABASE_PASSWORD`	Typing errors in the controller’s `local.conf`	Correction with `sed`, same four passwords on all nodes
10	`NO_PUBKEY` error: “The repository ... is not signed” during `apt update`	Tailscale repository signing key missing from the VM	Download of the official key into `/usr/share/keyrings`, then `apt update`
11	Wi-Fi IP addresses unusable after the network change	Different subnet, then machines on separate networks	Switch to Tailscale addresses in `/etc/hosts` and in the `local.conf` files
12	`bash` syntax error (`unexpected token newline`) when pasting a command	Angle brackets `< >` left around a value to replace, interpreted as redirections	Remove the brackets and enter the value alone

Table 6: Difficulties encountered and solutions adopted

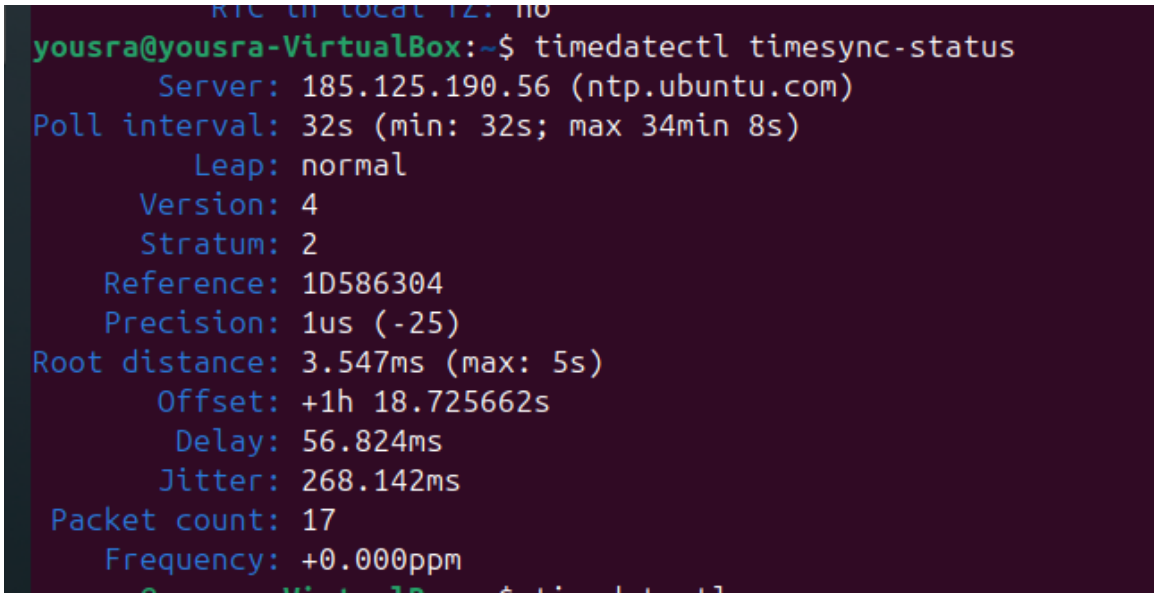


Figure 49: VM clock offset by one hour

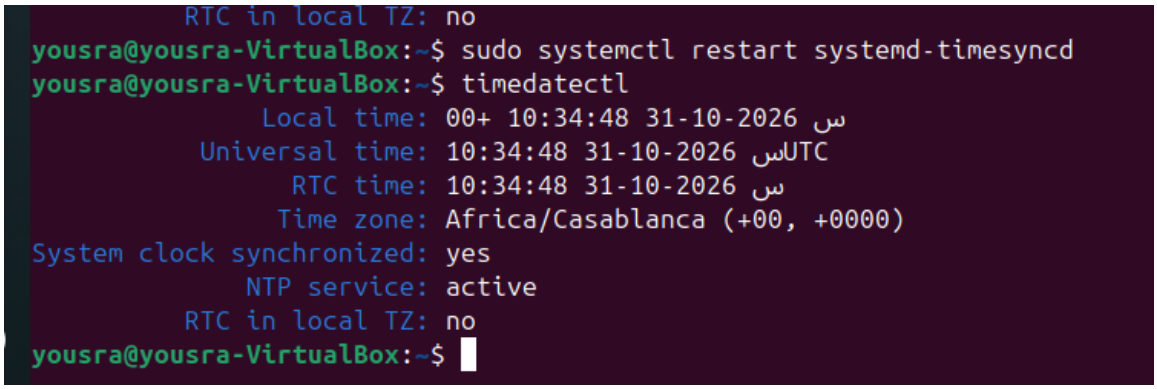


Figure 50: Clock synchronized after restarting `systemd-timesyncd`

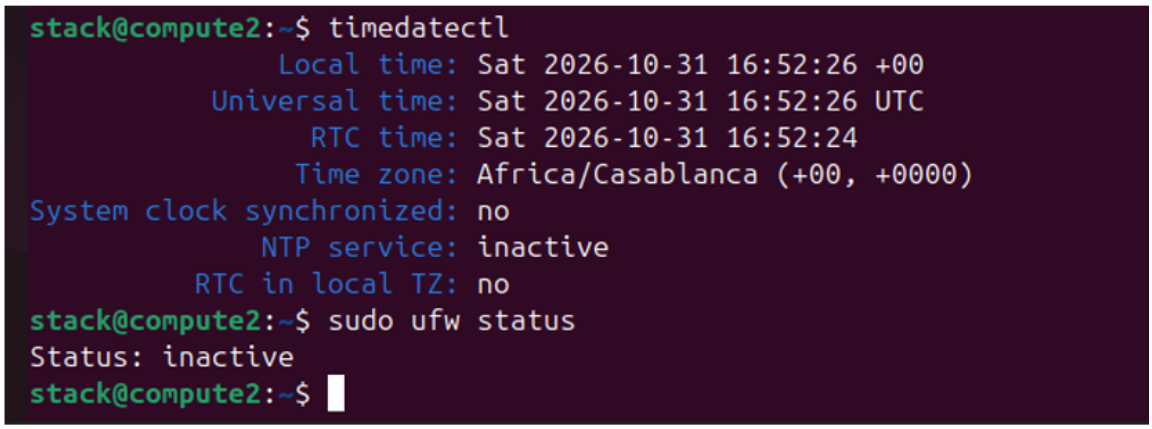


Figure 51: Clock not synchronized on `compute2` (NTP service inactive)

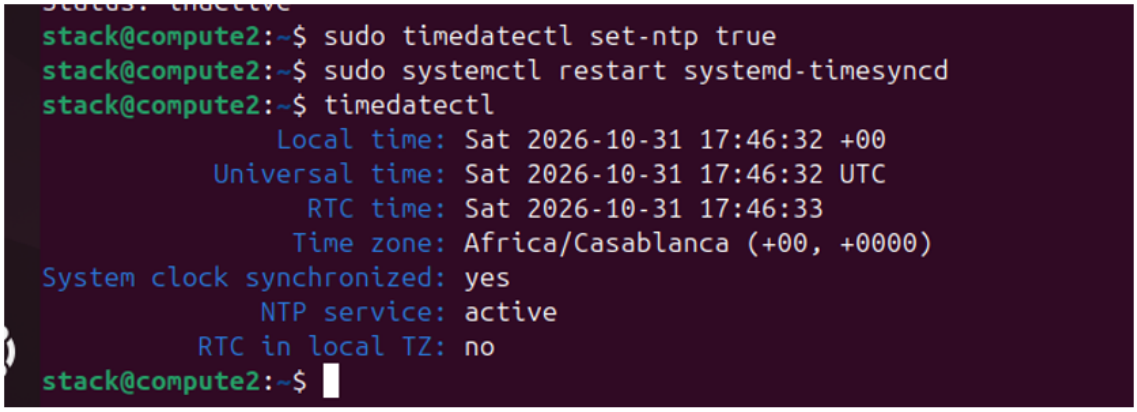


Figure 52: Clock synchronized on `compute2` after correction

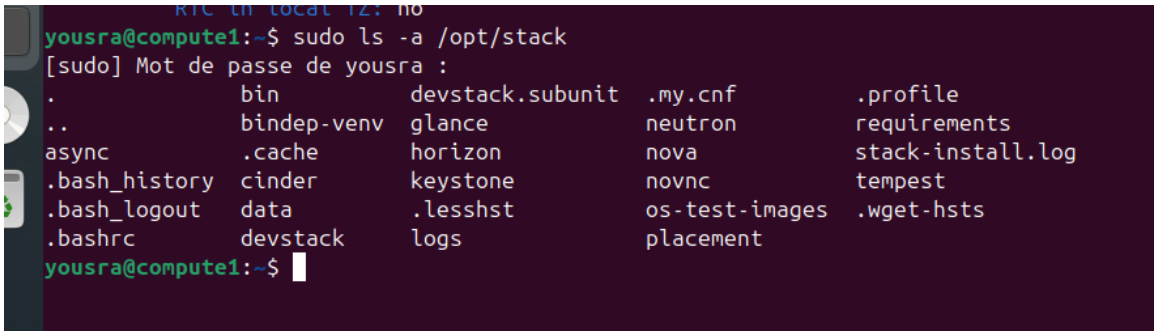


Figure 53: Leftovers of an old installation in `/opt/stack`

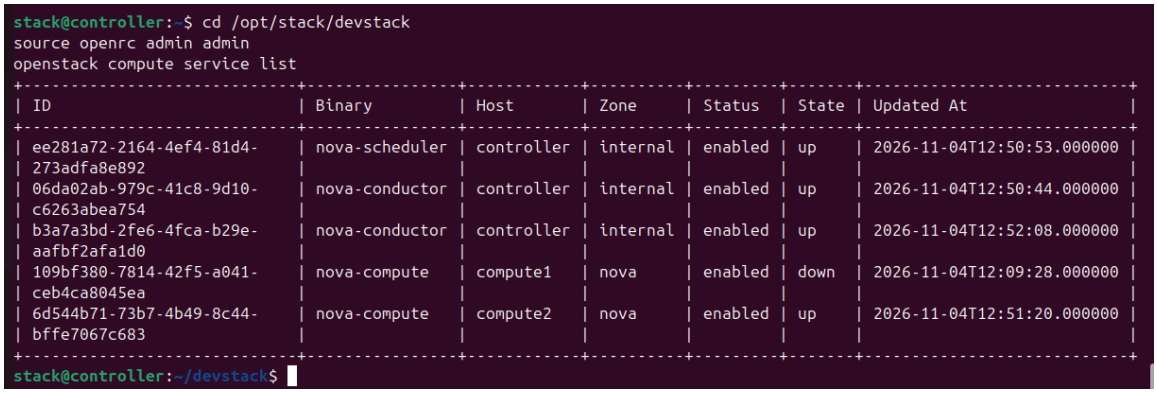


Figure 54: `compute1` in the `down` state and `compute2` in the `up` state in the compute service list

3 Part 3 – Creating and administering cloud resources

The resources in this part are created on the command line from the controller node, with the `stack` user and the administrator identity of the `admin` project (`openrc` file). Since the cloud is shared, a resource created by one team member is visible to the other two. Table 7 summarizes the resources created.

Resource	Name	Characteristics
Image	`cirros-0.6.3-x86_64-disk`	QCOW2, 21 692 416 bytes, status `active`
Flavor	`m1.test`	1 vCPU, 256 MB of RAM, 1 GB of disk
Private network	`tp-net`	MTU 1222, status `ACTIVE`
Subnet	`tp-subnet`	`192.168.50.0/24`, gateway `192.168.50.1`, DHCP enabled
Router	`tp-router`	Interface on `tp-subnet`, external gateway on `public`
Key pair	`mykey`	SSH type
Security rules	`default` group	Inbound ICMP and TCP/22
Instances	`test1`, `test2`	CirrOS image, `m1.test` flavor, `tp-net` network

Table 7: Cloud resources created

After loading the administrator identity (`source openrc admin admin`), the following commands were run:

```
openstack flavor create --vcpus 1 --ram 256 --disk 1 m1.test
openstack keypair create mykey > ~/mykey.pem && chmod 600 ~/mykey.pem
openstack security group rule create --proto icmp default
openstack security group rule create --proto tcp --dst-port 22 default
openstack network create tp-net
openstack subnet create --network tp-net --subnet-range 192.168.50.0/24 tp-subnet
openstack router create tp-router
openstack router add subnet tp-router tp-subnet
openstack router set --external-gateway public tp-router
```

```
-bash: syntax error near unexpected token `newline'
stack@controller:~$ openstack flavor create --vcpus 1 --ram 256 --disk 1 m1.test
openstack keypair create mykey > ~/mykey.pem && chmod 600 ~/mykey.pem
openstack security group rule create --proto icmp default
openstack security group rule create --proto tcp --dst-port 22 default
openstack network create tp-net
openstack subnet create --network tp-net --subnet-range 192.168.50.0/24 tp-subnet
openstack router create tp-router
openstack router add subnet tp-router tp-subnet
openstack router set --external-gateway public tp-router
+---------------------------+--------------------------------------+
| Field                     | Value                                |
```

Figure 55: Commands used to create the Part 3 resources

3.1 System image

An image is a virtual machine template: it is stored in Glance on the controller, then copied to the compute node that runs the instance. The CirrOS image `cirros-0.6.3-x86_64-disk`, a minimal Linux distribution suited to the software emulation we use, was imported into Glance by DevStack during the controller installation. It is public, in QCOW2 format, about 20.7 MB in size, with the status `active`. It can be seen with the following commands:

```
openstack image list
openstack image show cirros-0.6.3-x86_64-disk -c name -c status -c disk_format -c
    size
```

```
+---------------------+---------------------+
stack@controller:~$ openstack image list
openstack image show cirros-0.6.3-x86_64-disk -c name -c status -c disk_format -c size
+--------------------------------------+--------------------------+--------+
| ID                                   | Name                     | Status |
+--------------------------------------+--------------------------+--------+
| f1b50fed-bafd-45fe-b841-8351d0525a12 | cirros-0.6.3-x86_64-disk | active |
+--------------------------------------+--------------------------+--------+
+-------------+--------------------------+
| Field       | Value                    |
+-------------+--------------------------+
| disk_format | qcow2                    |
| name        | cirros-0.6.3-x86_64-disk |
| size        | 21692416                 |
| status      | active                   |
+-------------+--------------------------+
stack@controller:~$
```

Figure 56: CirrOS image in Glance (command line)

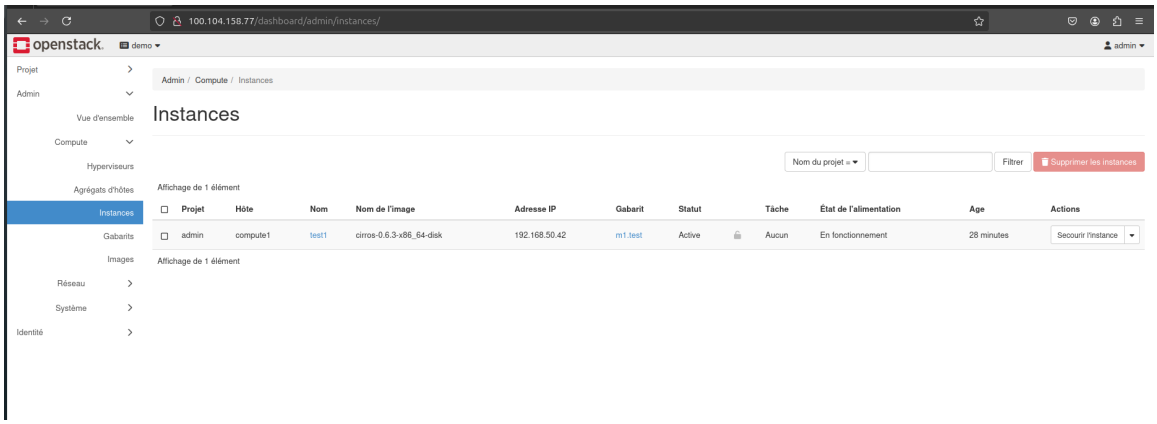


Figure 57: CirrOS image in Horizon

3.2 Instance type (flavor)

A *flavor* defines the resources of an instance. The `m1.test` flavor is deliberately minimal, because the instances run under software emulation: 1 virtual processor, 256 MB of memory and 1 GB of disk.

```
stack@controller:~$ openstack flavor show m1.test -c name -c vcpus -c ram -c disk
+-------+---------+
| Field | Value   |
+-------+---------+
| disk  | 1       |
| name  | m1.test |
| ram   | 256     |
| vcpus | 1       |
+-------+---------+
stack@controller:~$
```

Figure 58: The `m1.test` flavor

3.3 Private network and subnet

The private network `tp-net` and its subnet `tp-subnet` follow the addressing plan in Table 8. The OpenStack DHCP server automatically assigns addresses to the instances.

Parameter	Value
Network	`tp-net`
Subnet	`tp-subnet`
Network address (CIDR)	`192.168.50.0/24`
Gateway	`192.168.50.1`
Range of assigned addresses	`192.168.50.2` to `192.168.50.254`
DHCP	enabled
Network MTU	1222

Table 8: Addressing plan of the private network

The MTU of the private network is 1222 bytes. It corresponds to the VPN packet size (1280 bytes, the value given to Neutron on the controller) minus the encapsulation used between the nodes: the instances' packets must fit inside the Tailscale tunnel.

```
stack@controller:~$ openstack network show tp-net -c name -c status -c mtu -c subnets
openstack subnet show tp-subnet -c name -c cidr -c gateway_ip -c allocation_pools -c enable_dhcp
+---------+--------------------------------------+
| Field   | Value                                |
+---------+--------------------------------------+
| mtu     | 1222                                 |
| name    | tp-net                               |
| status  | ACTIVE                               |
| subnets | 81758f99-97ab-4e7f-a306-6603a26514ac |
+---------+--------------------------------------+
+------------------+----------------------------+
| Field            | Value                      |
+------------------+----------------------------+
| allocation_pools | 192.168.50.2-192.168.50.254 |
| cidr             | 192.168.50.0/24            |
| enable_dhcp      | True                       |
| gateway_ip       | 192.168.50.1               |
| name             | tp-subnet                  |
+------------------+----------------------------+
stack@controller:~$
```

Figure 59: Network `tp-net` and subnet `tp-subnet`

3.4 Virtual router and external access

The router `tp-router` connects the private subnet to the external network `public` created by DevStack. It has an interface on the private subnet (address `192.168.50.1`) and an external gateway on the `public` network (address `172.24.4.26`), with address translation (SNAT) enabled: the instances can therefore reach the external network. Its status is `ACTIVE`. Access from the outside to an instance goes through a floating IP address, used for the Part 4 tests. The `public` network is one of the networks created by DevStack, along with the `private` and `shared` networks.

```
stack@controller:~$ openstack router show tp-router -c name -c status -c external_gateway_info -c interfaces_info
+----------------------+-----------------------------------------------------------------------------------------------------+
| Field                | Value                                                                                               |
+----------------------+-----------------------------------------------------------------------------------------------------+
| external_gateway_info | {"network_id": "f6e0fedb-f331-4573-a03c-150018c5499b", "external_fixed_ips": [{"subnet_id":  |
|                      | "efb0e47d-1f6e-4371-ad8c-4cbe092370f0", "ip_address": "172.24.4.26"}], "enable_snat": true}  |
| interfaces_info      | [{"port_id": "143ecfb3-2566-4866-84cc-4b2503ec58d5", "ip_address": "192.168.50.1",           |
|                      | "subnet_id": "81758f99-97ab-4e7f-a306-6603a26514ac"}]                                               |
| name                 | tp-router                                                                                           |
| status               | ACTIVE                                                                                              |
+----------------------+-----------------------------------------------------------------------------------------------------+
```

Figure 60: The `tp-router` router

```
stack@controller:~$ openstack network list
+--------------------------------------+---------+--------------------------------------+
| ID                                   | Name    | Subnets                              |
+--------------------------------------+---------+--------------------------------------+
| 7a638303-6f30-4cbd-a1ee-14fb59525ed8 | private | 49a94470-14d7-4148-92b8-1aed116c27be |
| 96a1cfc1-9a21-4f6b-b8ea-e7bd52fd8885 | shared  | c6cd7130-e766-41b3-962a-a0a9c28dd4af |
| f6e0fedb-f331-4573-a03c-150018c5499b | public  | efb0e47d-1f6e-4371-ad8c-4cbe092370f0 |
+--------------------------------------+---------+--------------------------------------+
stack@controller:~$
```

Figure 61: Networks created by DevStack (`private`, `shared` and `public`)

3.5 SSH key pair

The key pair `mykey` (SSH type) was created by OpenStack. The public key is registered in Nova and injected into the instances at boot; the private key was saved in the file `mykey.pem`, protected by `600` permissions (read and write reserved for the owner). This private key must not be shared.

```
stack@controller:~$ openstack keypair show mykey -c name -c fingerprint -c type
ls -l ~/mykey.pem
+-------------+-------------------------------------------------+
| Field       | Value                                           |
+-------------+-------------------------------------------------+
| fingerprint | e2:cf:83:9c:97:14:b6:4c:18:89:39:53:4a:fe:6c:fb |
| name        | mykey                                           |
| type        | ssh                                             |
+-------------+-------------------------------------------------+
-rw------- 1 stack stack 388 Nov  4 10:22 /opt/stack/mykey.pem
```

Figure 62: Key pair `mykey` and protected private key file

3.6 Security rules

By default, a security group blocks inbound traffic to the instances. Two rules were added to the `default` group, used by our instances: one rule allowing the ICMP protocol (for `ping`) and one rule allowing TCP on port 22 (SSH). These rules allow all source addresses (`0.0.0.0/0`), which is acceptable in this test environment, reachable only through the VPN. The list of rules shows the SSH rule (inbound, port 22) as well as the ICMP rule.

```
openstack security group rule list 72cb74e3-f425-4ce4-884d-5df37348c4f2 \
  -c "IP Protocol" -c "IP Range" -c "Port Range" -c Direction
```

3.7 Launching instances

Two instances, `test1` and `test2`, were created with the following command (example for `test1`):

```
openstack server create --flavor m1.test --image cirros-0.6.3-x86_64-disk \
  --network tp-net --key-name mykey \
  --security-group 72cb74e3-f425-4ce4-884d-5df37348c4f2 test1
```

On creation, the instance goes through the `scheduling` and then `building` states: the controller chooses the compute node that will host it. Table 9 summarizes their state.

Instance	Node	Private address	Remark
`test1`	`compute1`	`192.168.50.42`	First creation (2026-11-04, 11:11 UTC), seen in Horizon
`test1`	`compute1`	`192.168.50.32`	Re-created instance: boot test and console log; in the `SHUTOFF` state on the evening of 5 November
`test1`	–	`192.168.50.143`	`ping` and SSH tests (floating address `172.24.4.138`)
`test1`	–	`192.168.50.94`	Stop, restart and delete test
`test2`	`compute1`	`192.168.50.158`	Created on 2026-11-04 at 15:49 UTC; in the `SHUTOFF` state on the evening of 5 November

Table 9: Instances created during the tests

The instance `test1` was deleted and then re-created between some series of tests, which explains the different private addresses and identifiers in the table. Horizon shows that the first instance runs on the node `compute1` (“Host” column), with the status “Active”, the power state “Running” and the task “None”. This confirms the complete chain: the controller receives the request, its scheduler chooses `compute1`, and that node downloads the image from Glance and then starts the instance with QEMU. The list of instances with their host (Figure 64) shows that `test1` and `test2` are both hosted by `compute1` on the evening of 5 November; Horizon later shows an instance on `compute2` (Section 4.6).

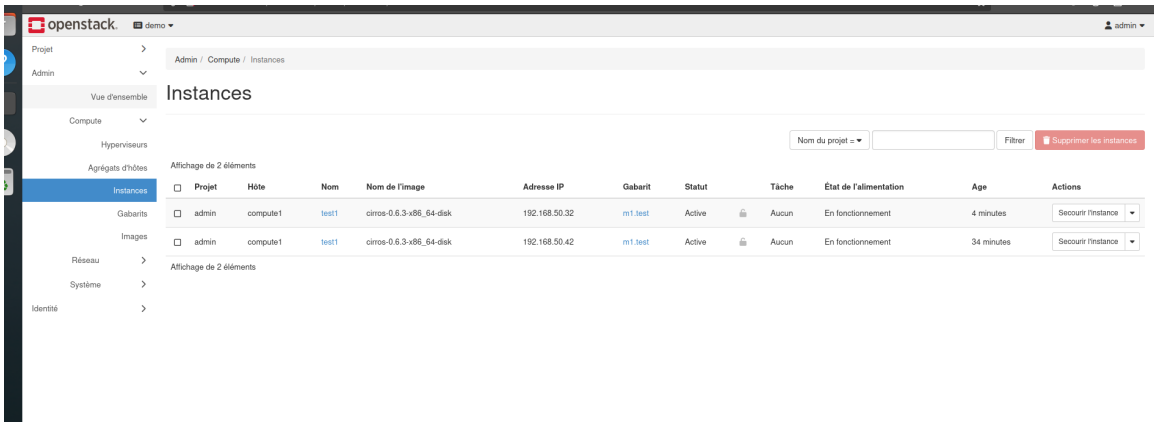


Figure 63: Instance `test1` running on the node `compute1`

```
dufei@compute2:~$ sudo -u stack bash -lc 'source /opt/stack/devstack/openrc admin admin && ope
nstack server list --long -c Name -c Status -c Host -c Networks'
+-------+---------+----------------------+----------+
| Name  | Status  | Networks             | Host     |
+-------+---------+----------------------+----------+
| test2 | SHUTOFF | tp-net=192.168.50.158 | compute1 |
| test1 | SHUTOFF | tp-net=192.168.50.32  | compute1 |
+-------+---------+----------------------+----------+
dufei@compute2:~$
```

Figure 64: Instances and the nodes that host them

3.8 Checking the state and resources of the instances

The state of an instance, its IP address and its flavor can be read with the following command:

```
openstack server show test1 -c name -c status -c addresses -c flavor \
  -c OS-EXT-SRV-ATTR:host -c key_name
```

On the node that hosts it, the instance is seen by the hypervisor under a name of the form `instance-0000000x`, with the command `sudo virsh list -all`, and it is run by a `qemu-system-x86_64` process. The CirrOS image is copied into the Nova cache of the compute node, which shows that it is the compute node, not the controller, that downloads the image. On `compute2`, which had not yet hosted any instance, this cache folder is empty (`total 0`, Figure 66): the image is downloaded only on the node that runs the instance. In Horizon, the “Instances” column of the hypervisor goes from 0 to 1.

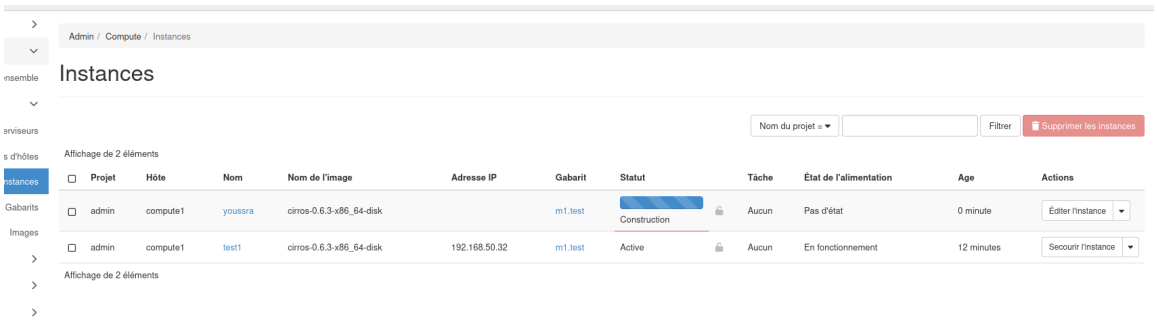


Figure 65: Instance seen by the hypervisor in the `compute1` VM

```
dufei@compute2:~$ sudo ls -lh /opt/stack/data/nova/instances/_base
total 0
dufei@compute2:~$
```

Figure 66: Empty image cache on `compute2` (no instance hosted)

4 Part 4 – Testing and validation

The tests are carried out from the controller node, with the instance `test1` created in the previous part. The floating IP address is taken from the external network `public` (`172.24.4.0/24`), reachable from the controller. Table 10 summarizes the tests and their results.

No.	Test	Result
1	Instance started	Status `ACTIVE`, host `compute1`, CirrOS boot log
2	Network connectivity	Floating address `172.24.4.138`, 4 `ping` replies out of 4
3	SSH connection	Successful connection with the key `mykey`
4	Stop, restart, delete	States `SHUTOFF`, `ACTIVE`, then instance deleted
5	Resource monitoring	One instance visible on `compute2` in Horizon

Table 10: Summary of the validation tests

4.1 Instance started correctly

The state of the instance and its boot log (console) are consulted with the following commands:

```
openstack server show test1 -c name -c status -c power_state -c addresses \
  -c OS-EXT-SRV-ATTR:host -c OS-EXT-SRV-ATTR:hypervisor_hostname
openstack console log show test1 | tail -15
```

The instance has the status `ACTIVE`; it runs on the host and hypervisor `compute1`, with the address `192.168.50.32` on the `tp-net` network. The console log shows that CirrOS 0.6.3 has booted and displays the login prompt (`test1 login:`), which confirms that the instance's system is operational.

```
 OS-EXT-SRV-ATTR:hypervisor_hostname^C
stack@controller:~$ openstack server show test1 -c name -c status -c power_state -c addresses -c OS-EXT-SRV-ATTR:host -
 OS-EXT-SRV-ATTR:hypervisor_hostname
openstack console log show test1 | tail -15
+-------------------------------------+---------------------+
| Field                               | Value               |
+-------------------------------------+---------------------+
| OS-EXT-SRV-ATTR:host                | compute1            |
| OS-EXT-SRV-ATTR:hypervisor_hostname | compute1            |
| addresses                           | tp-net=192.168.50.32 |
| name                                | test1               |
| status                              | ACTIVE              |
+-------------------------------------+---------------------+
instance-id: i-00000002
name: N/A
availability-zone: nova
local-hostname: test1.novalocal
launch-index: 0
=== cirros: current=0.6.3 uptime=35.16 ===
  ____               ____  ____
 / __/ __ ____ ____ / __ \/ __/
/ /__ / // __// __// /_/ /\ \
\___//_//_/  /_/   \____/___/
   http://cirros-cloud.net


login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
test1 login:
stack@controller:~$
```

Figure 67: State of the instance `test1` and CirrOS boot log

4.2 Network connectivity test

To reach the instance from outside its private network, a floating IP address is created on the `public` network, then associated with the instance:

```
openstack floating ip create public
openstack server add floating ip test1 172.24.4.138
openstack server list
ping -c 4 172.24.4.138
```

The list of instances shows the floating address `172.24.4.138` in addition to the private address of `test1`. The `ping` from the controller receives 4 replies out of 4, with no packet loss. The TTL of 63 indicates that the packets cross the virtual router, and the average delay of about 173 ms is probably explained by the software emulation (QEMU) and by the passage through the VPN.

```
stack@controller:~$ openstack floating ip create public
+---------------------+--------------------------------------+
| Field               | Value                                |
+---------------------+--------------------------------------+
| created_at          | 2026-11-06T10:03:47Z                 |
| description         |                                      |
| dns_domain          |                                      |
| dns_name            |                                      |
| fixed_ip_address    | None                                 |
| floating_ip_address | 172.24.4.138                         |
| floating_network_id | f6e0fedb-f331-4573-a03c-150018c5499b |
| id                  | 5b6f8510-7fa2-4381-976f-e357653fd1dd |
| name                | 172.24.4.138                         |
| port_details        | None                                 |
| port_id             | None                                 |
| project_id          | b035b3ef8af146b5a7ef7173a4867913     |
| qos_policy_id       | None                                 |
| revision_number     | 0                                    |
| router_id           | None                                 |
| status              | DOWN                                 |
| subnet_id           | None                                 |
| tags                | []                                   |
| updated_at          | 2026-11-06T10:03:47Z                 |
+---------------------+--------------------------------------+
stack@controller:~$ openstack server add floating ip test1 172.24.4.138
stack@controller:~$ openstack server list
+--------------------------------------+-------+--------+----------------------------+-------------------------+---------+
| ID                                   | Name  | Status | Networks                   | Image                   | Flavor  |
+--------------------------------------+-------+--------+----------------------------+-------------------------+---------+
| 77b23258-a0b9-4725-9587-195719b40a64 | test1 | ACTIVE | tp-net=172.24.4.138,       | cirros-0.6.3-x86_64-disk | m1.test |
|                                      |       |        | 192.168.50.143             |                         |         |
+--------------------------------------+-------+--------+----------------------------+-------------------------+---------+
```

Figure 68: Creation and association of the floating IP address

```
stack@controller:~$ ip neigh show 172.24.4.138
ping -c 4 172.24.4.138
172.24.4.138 dev br-ex lladdr fa:16:3e:00:1e:d5 REACHABLE
PING 172.24.4.138 (172.24.4.138) 56(84) bytes of data.
64 bytes from 172.24.4.138: icmp_seq=1 ttl=63 time=142 ms
64 bytes from 172.24.4.138: icmp_seq=2 ttl=63 time=348 ms
64 bytes from 172.24.4.138: icmp_seq=3 ttl=63 time=95.7 ms
64 bytes from 172.24.4.138: icmp_seq=4 ttl=63 time=107 ms

--- 172.24.4.138 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 95.743/173.295/348.294/102.503 ms
stack@controller:~$
```

Figure 69: `ping` test to the instance's floating address

4.3 SSH connection to the instance

The connection is made with the private key `mykey`, using the default CirrOS user:

```
ssh -i ~/mykey.pem cirros@172.24.4.138
```

On the first connection, SSH warns that the host key is unknown; it is accepted and saved. Inside the instance, the `hostname` command returns `test1`, and the `eth0` interface has the address `192.168.50.143/24` with an MTU of 1222 bytes, the value of the `tp-net` network. A `ping` to the gateway `192.168.50.1` (the virtual router) receives 3 replies out of 3. The SSH security rule and the key injected by OpenStack therefore work.

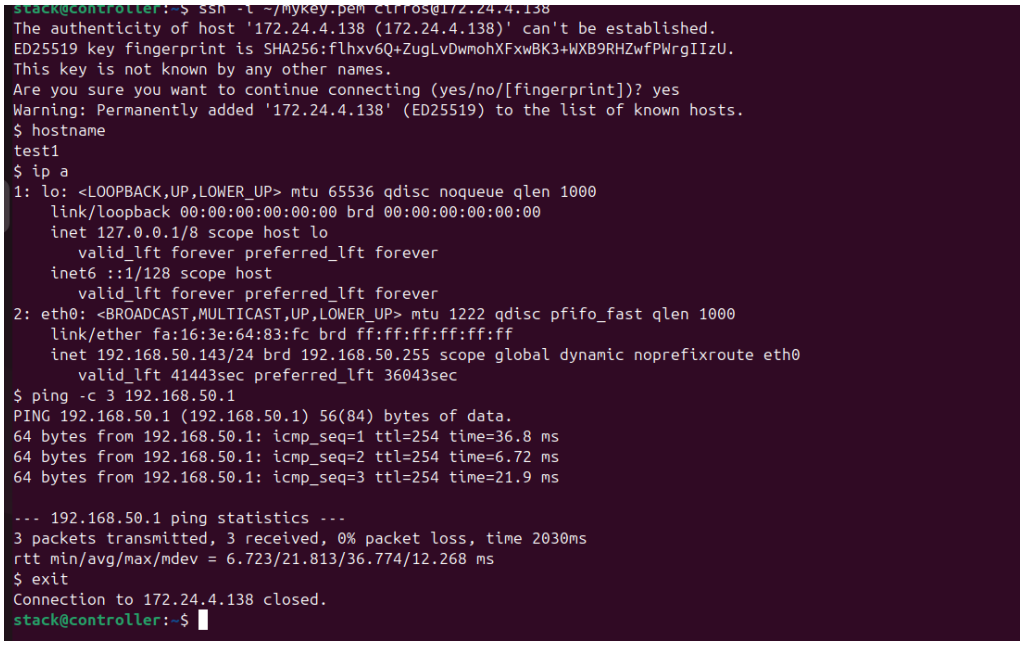


Figure 70: SSH connection to the instance `test1` through its floating address

4.4 Enforcement of the security rules

To show that the security rules are really enforced, we compare the behavior of the instance before and after removing the ICMP rule from the `default` group:

```
openstack security group rule delete ID_REGLE_ICMP
ping -c 3 172.24.4.138
openstack security group rule create --proto icmp default
ping -c 3 172.24.4.138
```

4.5 Stopping, restarting and then deleting an instance

The life cycle of a test instance is run with the following commands, listing the instances between each step:

```
openstack server stop test1
openstack server start test1
openstack server delete test1
openstack server list
```

The instance goes from the `ACTIVE` state to `SHUTOFF` after the stop. After the start command, it first still appears as `SHUTOFF`, then `ACTIVE` a few seconds later. After the deletion, the list briefly shows it with the state `DELETED`.

```
stack@controller:~$ openstack server list
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
| ID                                   | Name  | Status | Networks            | Image                   | Flavor  |
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
| c1c55351-b6a4-4bd9-8544-f4d951a9d1ef | test1 | ACTIVE | tp-net=192.168.50.94 | cirros-0.6.3-x86_64-disk | m1.test |
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
stack@controller:~$ openstack server stop test1
stack@controller:~$ openstack server list
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
| ID                                   | Name  | Status  | Networks            | Image                   | Flavor  |
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
| c1c55351-b6a4-4bd9-8544-f4d951a9d1ef | test1 | SHUTOFF | tp-net=192.168.50.94 | cirros-0.6.3-x86_64-disk | m1.test |
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
stack@controller:~$ openstack server start test1
stack@controller:~$ openstack server list
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
| ID                                   | Name  | Status  | Networks            | Image                   | Flavor  |
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
| c1c55351-b6a4-4bd9-8544-f4d951a9d1ef | test1 | SHUTOFF | tp-net=192.168.50.94 | cirros-0.6.3-x86_64-disk | m1.test |
+--------------------------------------+-------+---------+---------------------+-------------------------+---------+
stack@controller:~$ openstack server list
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
| ID                                   | Name  | Status | Networks            | Image                   | Flavor  |
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
| c1c55351-b6a4-4bd9-8544-f4d951a9d1ef | test1 | ACTIVE | tp-net=192.168.50.94 | cirros-0.6.3-x86_64-disk | m1.test |
+--------------------------------------+-------+--------+---------------------+-------------------------+---------+
stack@controller:~$ openstack server delete test1
stack@controller:~$ openstack server list
+--------------------------------------+-------+---------+----------+-------------------------+---------+
| ID                                   | Name  | Status  | Networks | Image                   | Flavor  |
+--------------------------------------+-------+---------+----------+-------------------------+---------+
| c1c55351-b6a4-4bd9-8544-f4d951a9d1ef | test1 | DELETED |          | cirros-0.6.3-x86_64-disk | m1.test |
+--------------------------------------+-------+---------+----------+-------------------------+---------+
```

Figure 71: Stopping, restarting and deleting the instance `test1`

4.6 Resource monitoring

The Horizon page *Admin* → *Compute* → *Hypervisors* summarizes the resources of the two compute nodes. `compute1` shows 0 vCPU used and 15.6 GB of memory available. `compute2` shows 1 vCPU used, 256 MB of memory used out of 9.7 GB and 1 GB of disk used out of 48 GB: these values match the `m1.test` flavor (1 vCPU, 256 MB, 1 GB), which shows that an instance runs on `compute2`. The total memory of the cloud is 25.3 GB, the sum of that of the two nodes. The vCPU capacity is higher than the number of processors of the virtual machines, because Nova applies an overcommit ratio.

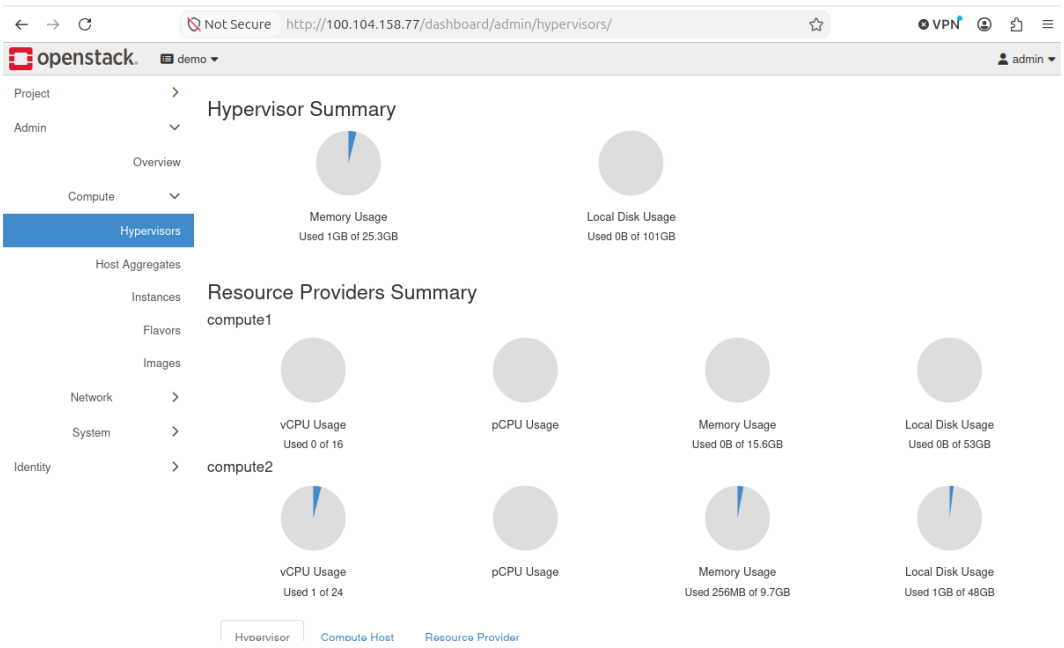


Figure 72: Resources of the two compute nodes in Horizon

5 Conclusion

This workshop allowed us to build a multi-node OpenStack private cloud, then to use it as a user would.

Summary

Using DevStack (`stable/2026.1` branch) on Ubuntu 24.04, we deployed one controller node and two compute nodes, each being a virtual machine on the PC of a team member. The three nodes are connected by a Tailscale virtual private network. The controller hosts Keystone, Nova, Neutron, Glance, Placement and Horizon; the compute nodes run the instances with QEMU. The compute services and the hypervisors are in the `up` state, and Horizon is accessible from any VM on the VPN.

In Part 3, we created the requested resources: CirrOS image, `m1.test` flavor, private network `tp-net` and its subnet, router `tp-router`, key pair `mykey`, ICMP and SSH security rules, and instances. The tests of Part 4 validated that everything works: instance started (status `ACTIVE`), reachable through a floating IP address (`ping` with no loss), SSH connection with the key, stop, restart and delete cycle, and resources visible in Horizon on both compute nodes.

Difficulties and limitations

- **Network:** Wi-Fi proved poorly suited to a cloud spread over three PCs: insufficient bandwidth, blocked ping, addresses tied to the subnet. The Tailscale VPN provided a stable common network, at the cost of a 1280-byte MTU (1222-byte private networks) and higher delays, around 160 ms, sometimes going through a relay.
- **Software emulation:** without hardware virtualization in the VMs, the instances run under QEMU and remain slow. We therefore chose a lightweight image (CirrOS) and a minimal flavor.
- **Scope and resources:** the disks of our VMs are smaller than the official recommendation, and Cinder is not deployed, because the practical part of the assignment does not need it. DevStack remains a tool for learning and testing, not intended for production.
- **Reliability:** frozen downloads required timeouts (`GIT_TIMEOUT`, `PIP_DEFAULT_TIMEOUT`), and the `nova-compute` service on `compute1` temporarily went into the `down` state.

Lessons learned

- **Architecture:** the controller makes decisions and stores the images, while the compute node downloads the image and runs the instance. Separating these roles shows how a cloud gains capacity by adding compute nodes.
- **Network:** in a multi-node deployment, the network is the most sensitive point: stable addresses, name resolution, packet size and Internet access must be checked before installation.
- **Method:** checking each step (service status, logs, connection tests) and documenting errors with their solutions allowed us to recover quickly after each failure.
- **Teamwork:** each member prepared, installed and verified a node, which forced us to agree on versions, addresses and shared passwords.

Outlook

On machines that expose hardware virtualization, enabling KVM would greatly speed up the instances. The Cinder service could be added to work with persistent volumes, and the security tests could be extended (separate security groups, systematic comparison before and after the rules). Finally, a deployment closer to production could rely on a tool such as Kolla-Ansible.

References

[1] Official OpenStack documentation, `https://docs.openstack.org/`.

[2] DevStack guide, “Multi-Node Lab”, `https://docs.openstack.org/devstack/latest/guides/multinode-lab.html`.

[3] OpenStack Nova documentation, installing a compute node on Ubuntu (page for version 2026.2, accessed as a complement), `https://docs.openstack.org/nova/2026.2/install/compute-install-ubuntu.html`.